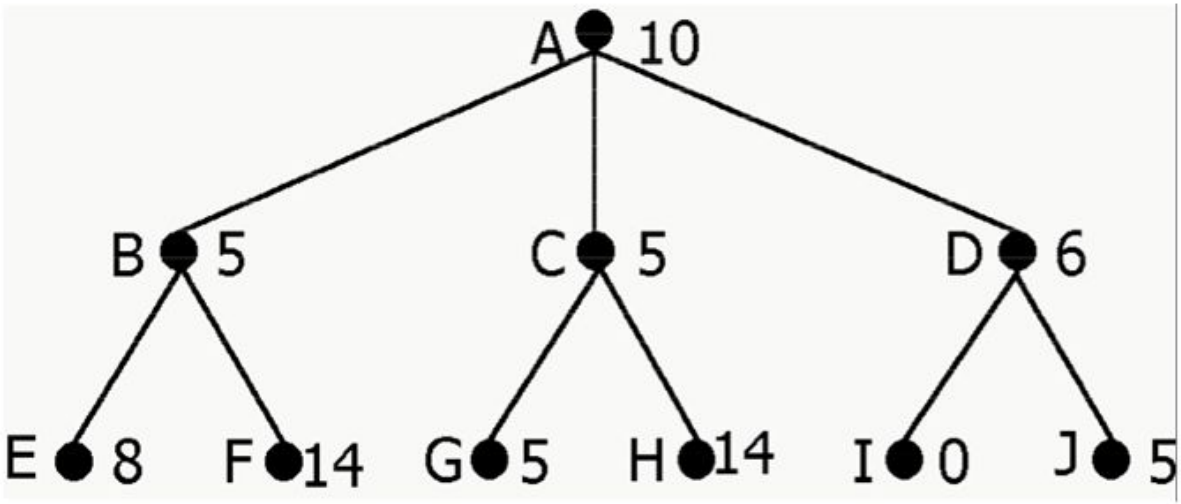
WG 2

Intelligent Systems

Exercise 2: Consider the following state-space, in which node I is a goal node. Furthermore, the lower the heuristic values, the better.



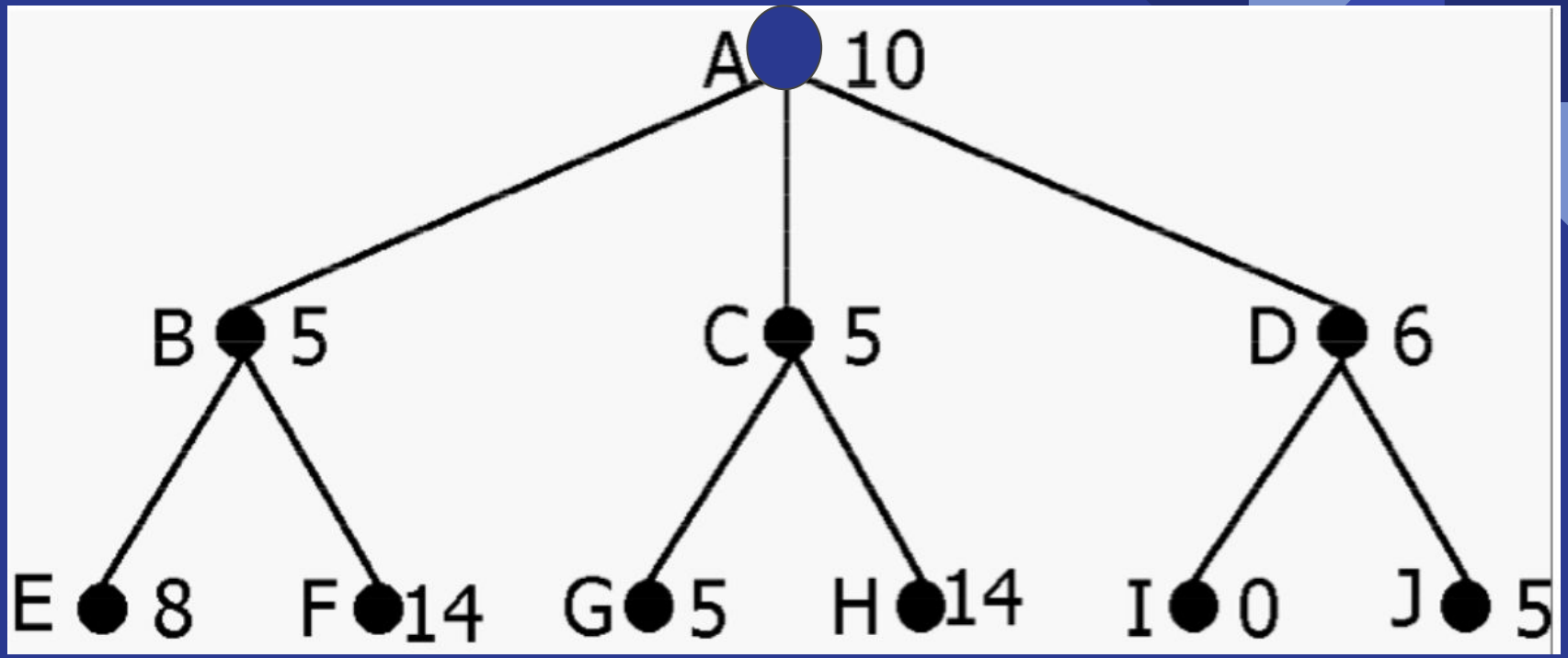
a) What is the order in which the nodes are searched by using **hill climbing**? Write down the letters of the nodes. Choose in alphabetical order when necessary.

b) What is the order in which the nodes are searched by using **greedy best-first search**?

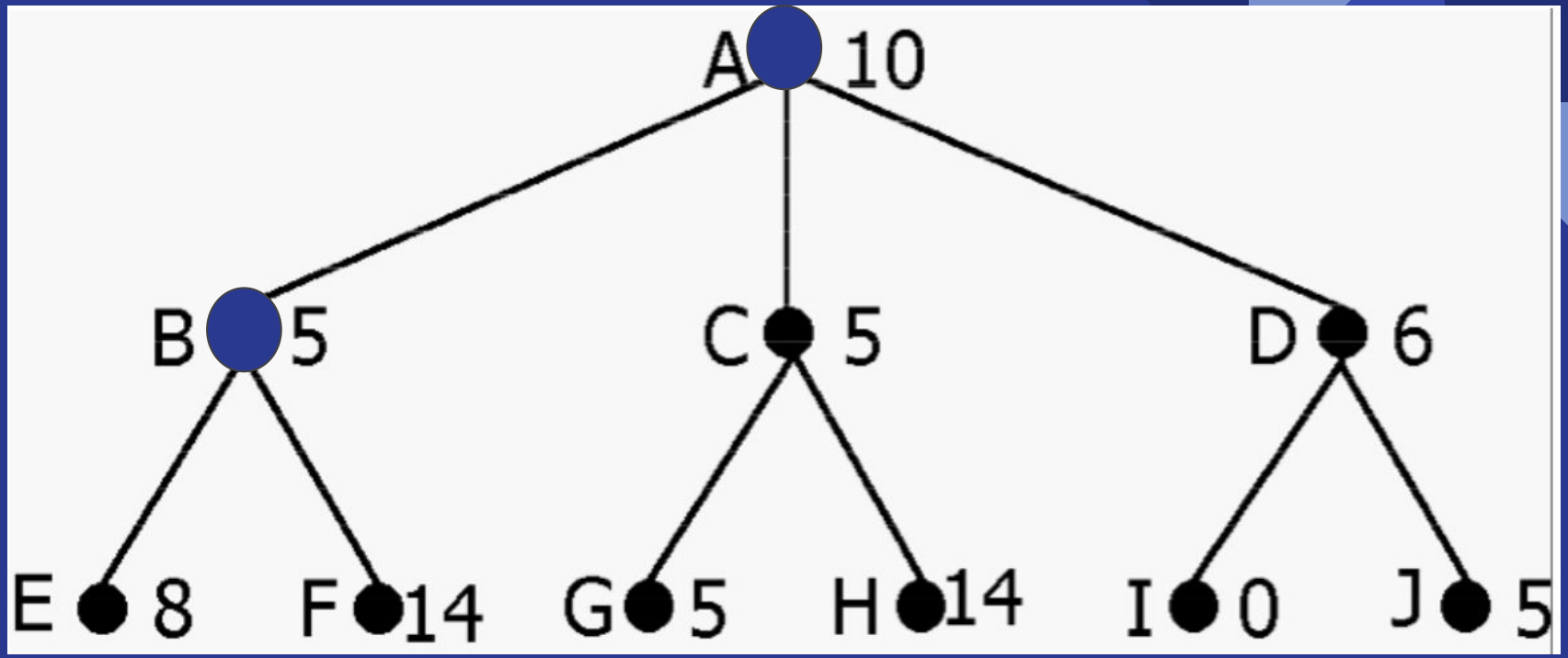
HILL CLIMBING

- Iterative algorithm
- Starts with an arbitrary solution to a problem
- Attempts to find a better solution by incrementally changing a single element of the solution.
- If the change produces a better solution, an incremental change is taken as the new solution, and the process repeats until no further improvements can be made.

For the given graph, the nodes have values which represent a heuristic that estimates the closest distance to the goal (lower values are typically better). In hill climbing, you always move to the adjacent node with the best value.



Answer : A



Answer : A B

2.a solution

a) What is the order in which the nodes are searched by using **hill climbing**? Write down the letters of the nodes. Choose in alphabetical order when necessary. AB. Once B is reached there is no node in the fringe with a higher value of h.

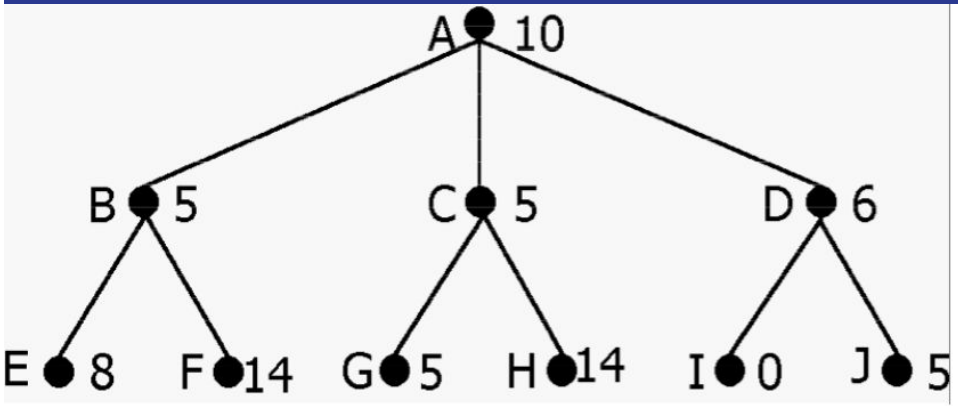
WHY?

- Start at node A.
- The next possible nodes to visit are B, C, and D.
- According to the hill climbing approach, you would choose the next node with the best value. B has a value of 5, C has a value of 5, and D has a value of 6.
- However, since we need to choose in alphabetical order when necessary, between B and C (both with the same value of 5), we choose B.
- Once B is reached, there are no adjacent nodes (E and F) with a better value than B.
- The algorithm stops here because it cannot move to a node with a better value.

GREEDY BEST-FIRST SEARCH

- Heuristic-driven search algorithm
- Expands the node that, as estimated by a heuristic function, is closest to the goal
- Chooses next step based solely on the heuristic, ignoring the path cost to that point.

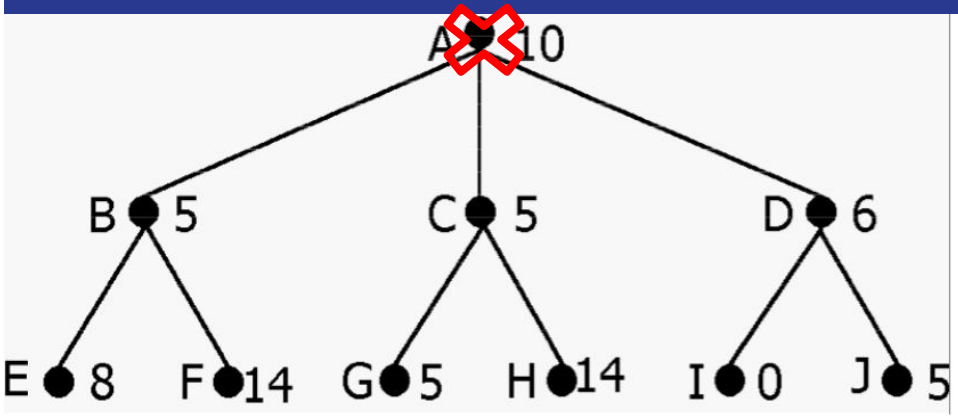
A



D

C

B



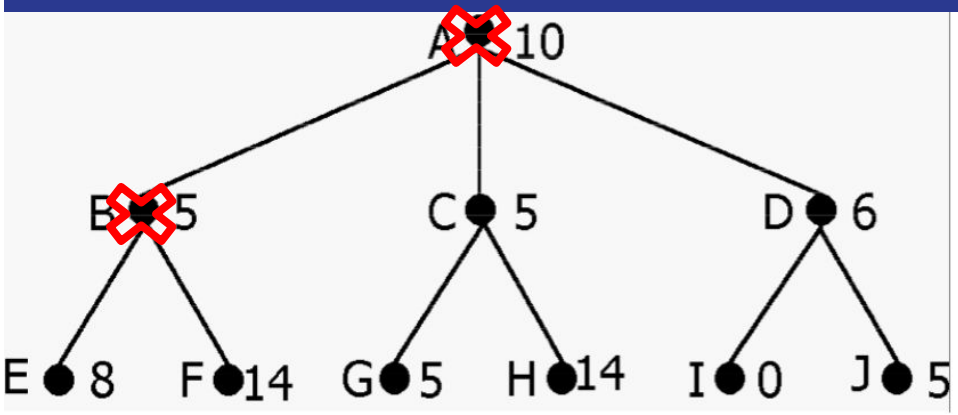
Answer: A

F

E

D

C



Answer: A B

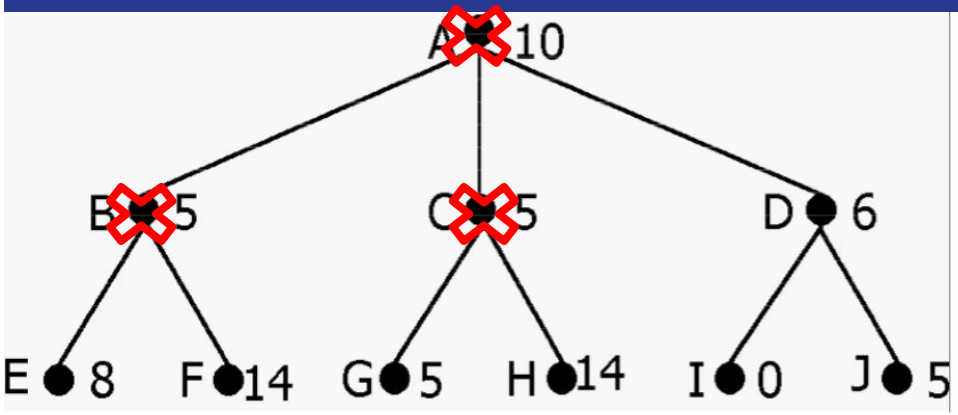
H

F

E

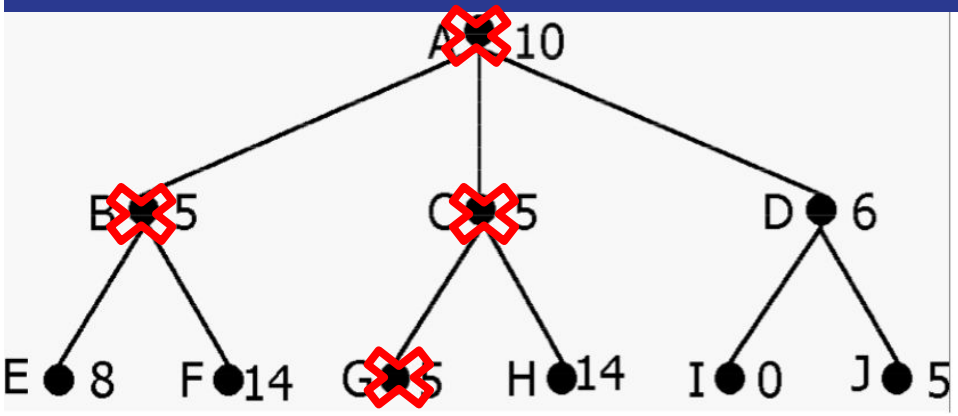
D

G



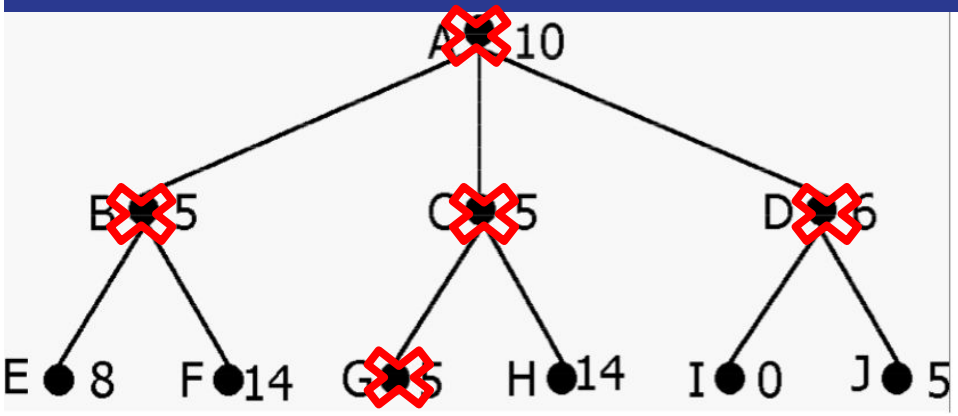
Answer: A B C

H
F
E
D



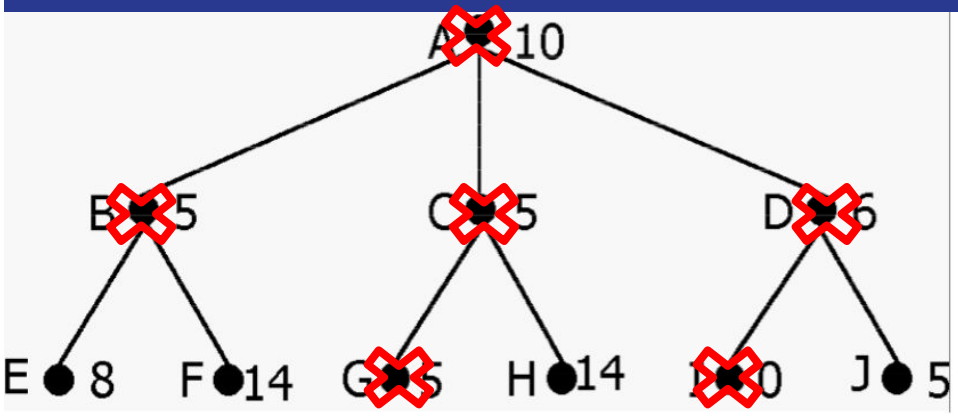
Answer: A B C G

H
F
E
J
I



Answer: A B C G D

H
F
E
J



Answer: A B C G D I

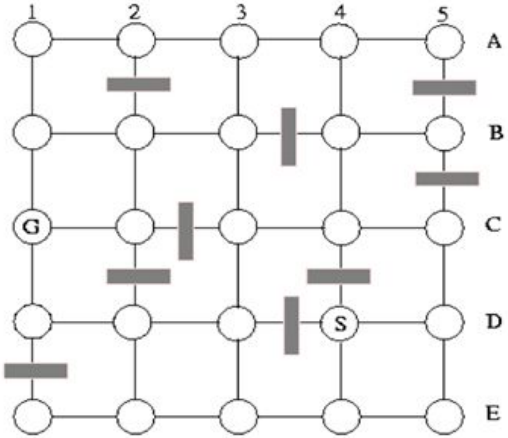
2.b solution

b) What is the order in which the nodes are searched by using **greedy best-first search**? Choose in alphabetical order when necessary. ABCGDI

WHY?

- Start at node A and push it onto the fringe.
- The possible nodes to visit are B, C, and D. Add these to the fringe.
- Choose the next node from the fringe with the lowest heuristic value.
- The node with the lowest heuristic value is B (5), but since B does not lead to the goal, the next nodes are added: E and F.
- The next node with the lowest heuristic value is C (5), which is selected, and the next node G and H added to the fringe.
- From the available nodes (G, D, E, F, H), G has the lowest heuristic value (5), but it is not the goal.
- D is chosen next (with a heuristic value of 6), leading to I and J being added to the fringe.
- I has the lowest heuristic value (0), indicating it is the goal, so the search stops here with the solution found.

Exercise 3: Given the following maze:



Goal of the maze is to search a route from start position D4 (marked 'S') to goal position C1 (marked 'G').

Draw for this maze the search tree by applying a data-driven A* algorithm that uses the Manhattan distance as heuristic h(n). This means that h(n) is just an addition of the horizontal and vertical distances, without taking into account the walls. Which path will eventually be searched by A*? Number the nodes in visiting order by the algorithm. Previously searched nodes with equal or higher values can be ignored. Furthermore, write down the used heuristic value of each node. Use alphabetical order when you encounter equal values.

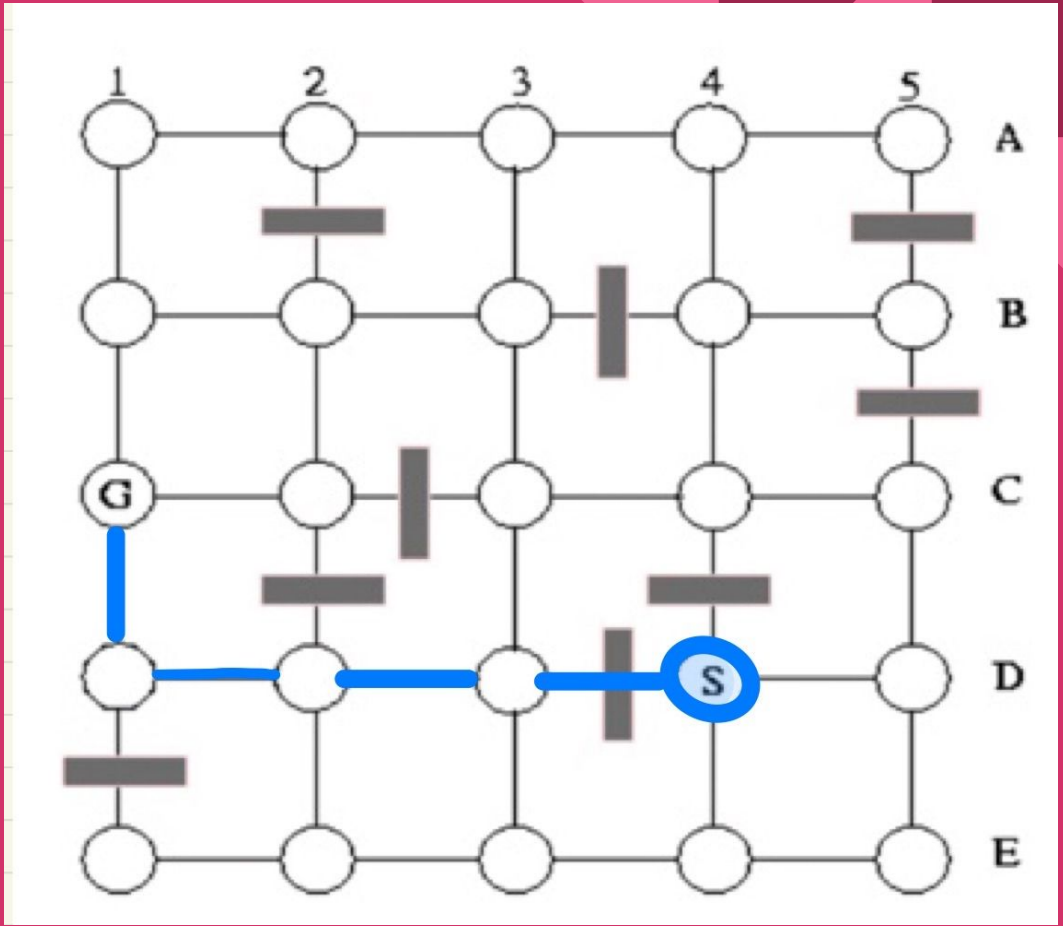
A* ALGORITHM – Manhattan Distance

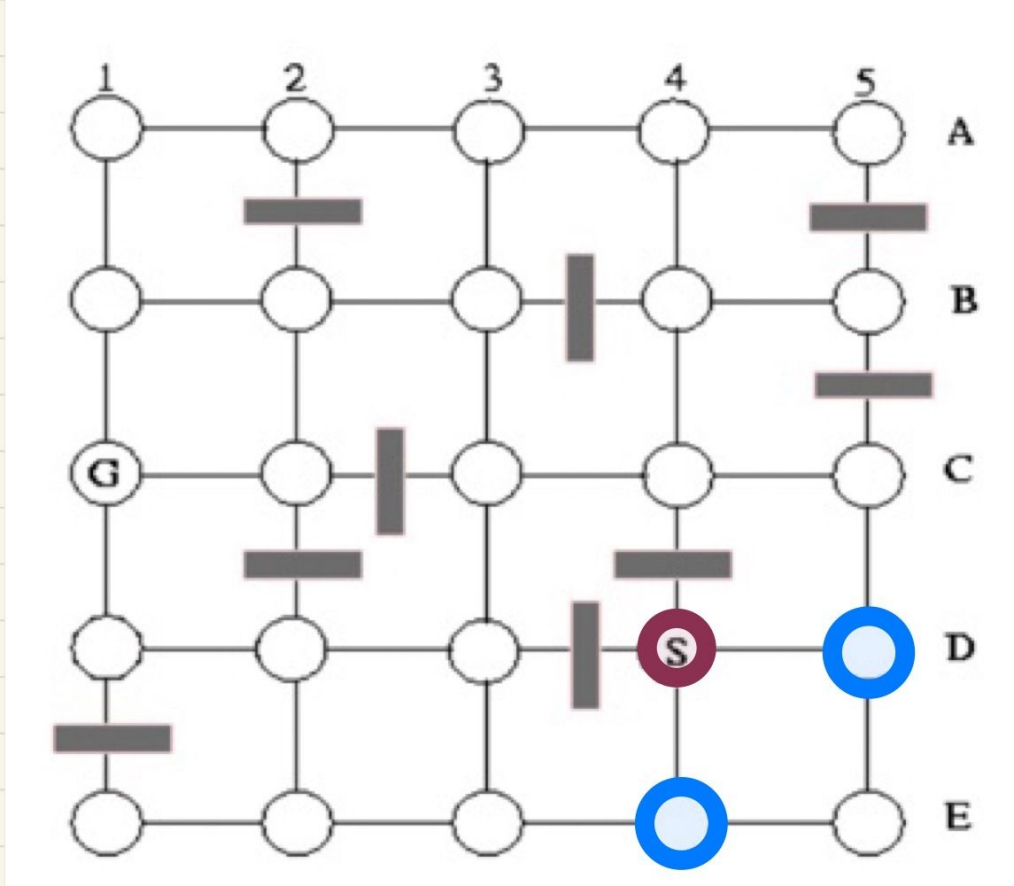
$$f(n) = g(n) + h(n)$$

g(n) = actual cost from start node to n

h(n) = heuristic estimate of the cost from n to the goal.
With Manhattan Distance, h(n) is the sum of the vertical and horizontal distances

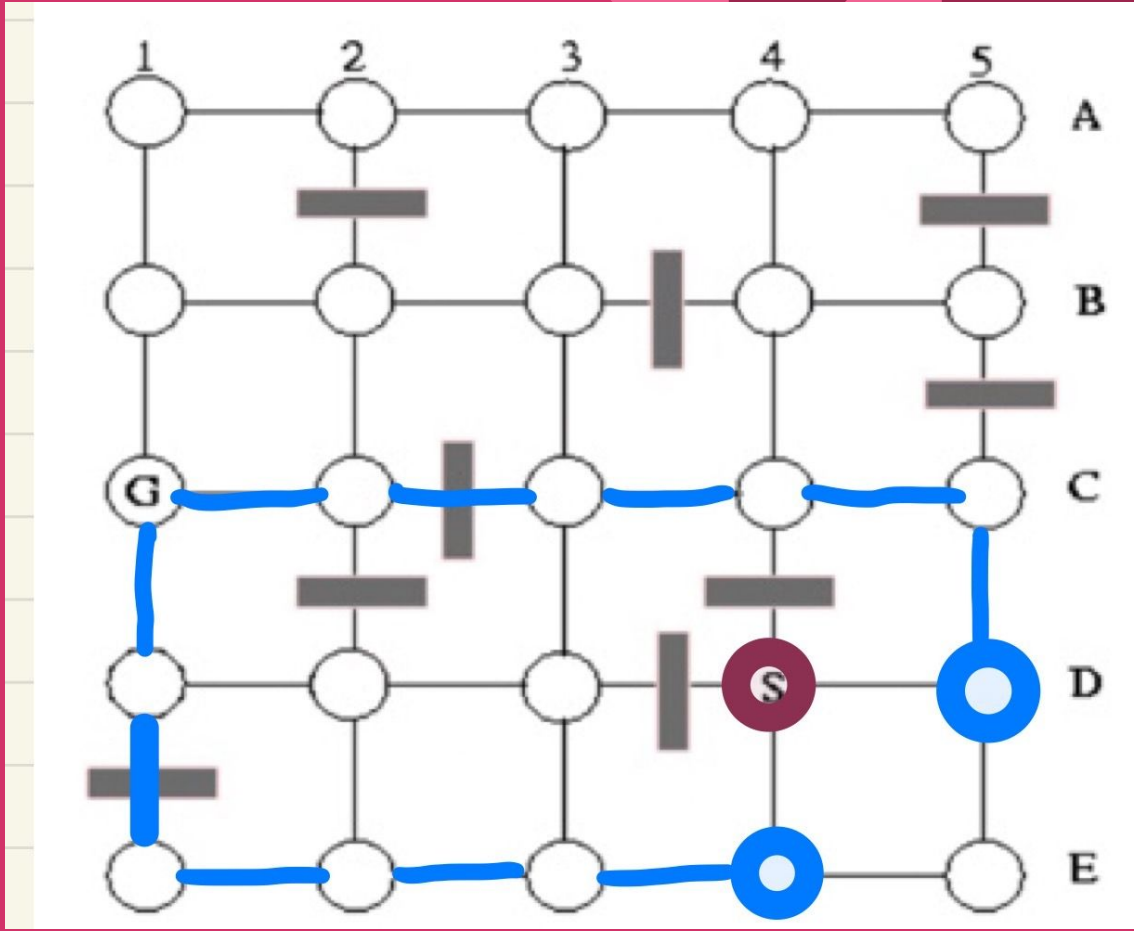
D4 = 0 + 4

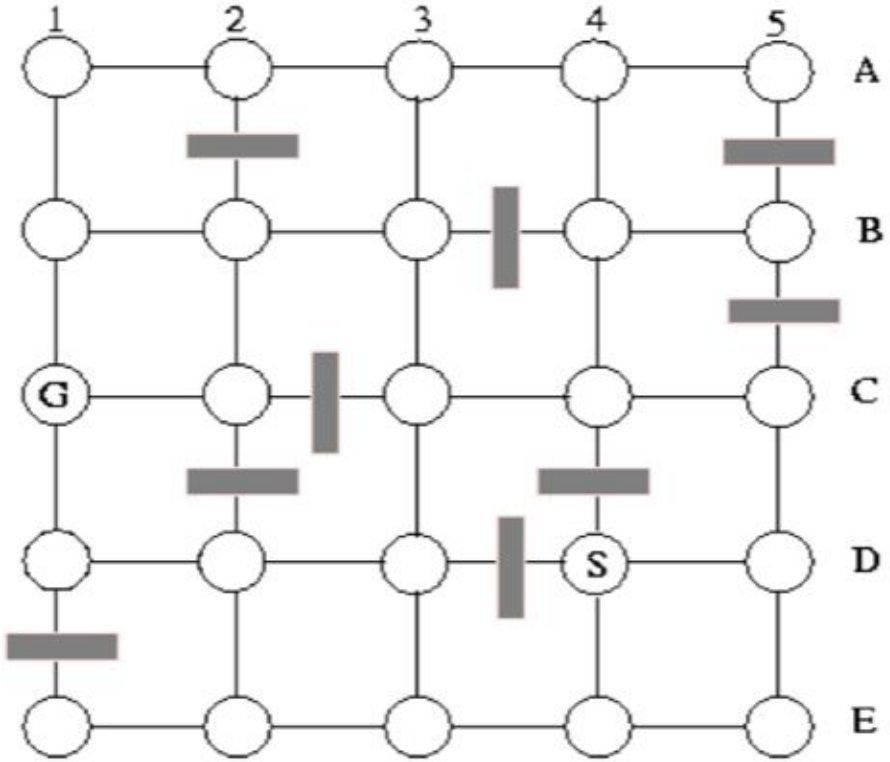


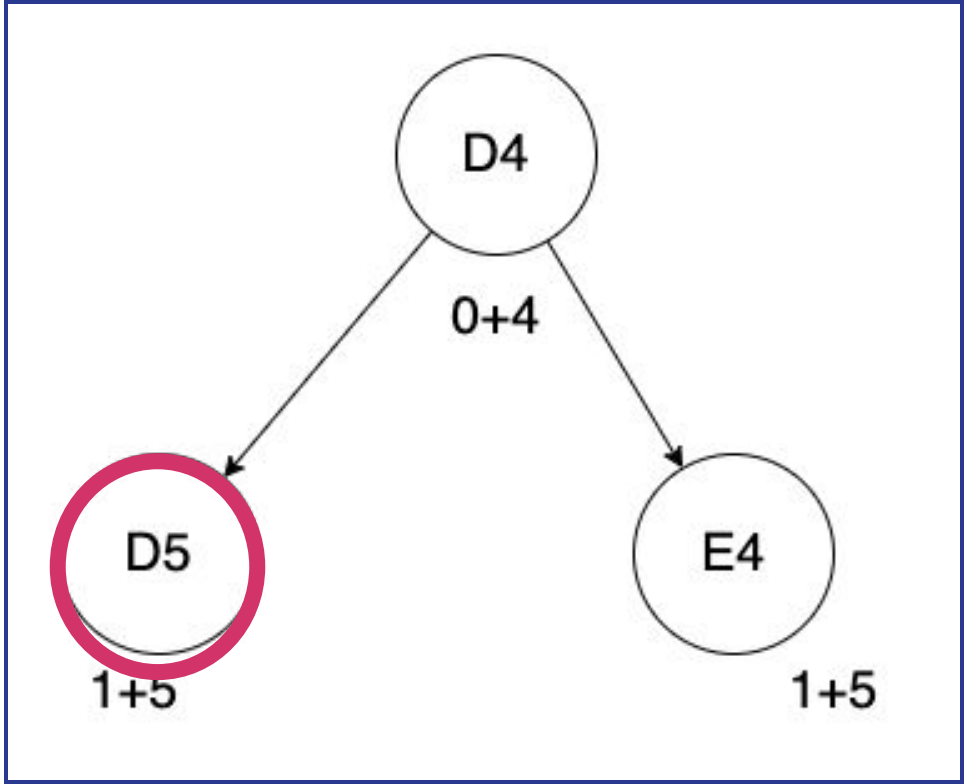

1
2
3
4
5
A
B
C
D
E
G
S

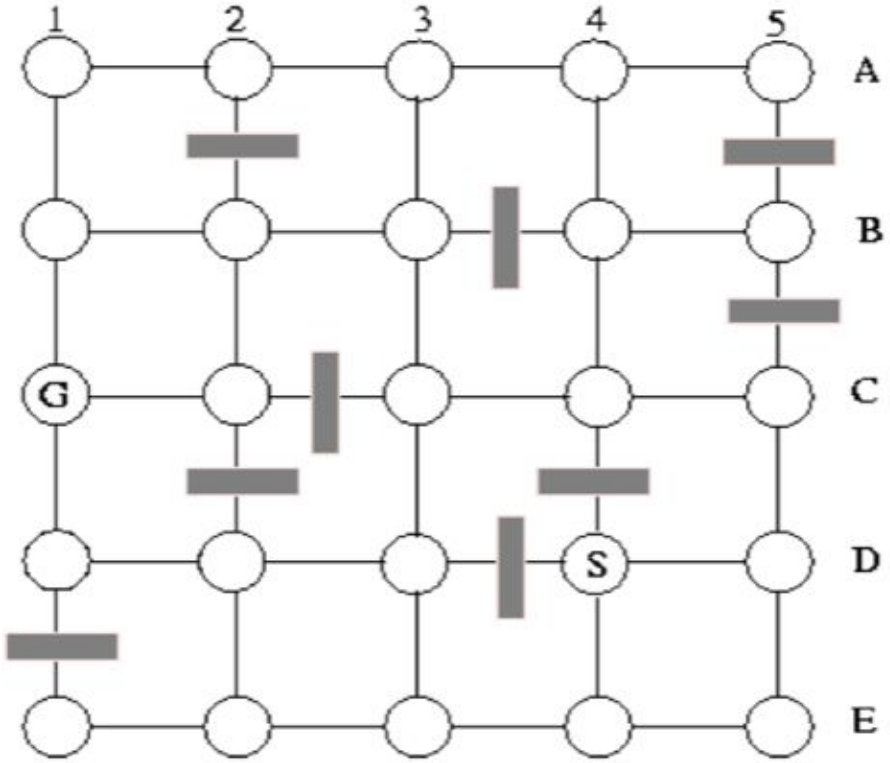
E4 = 1 + 5

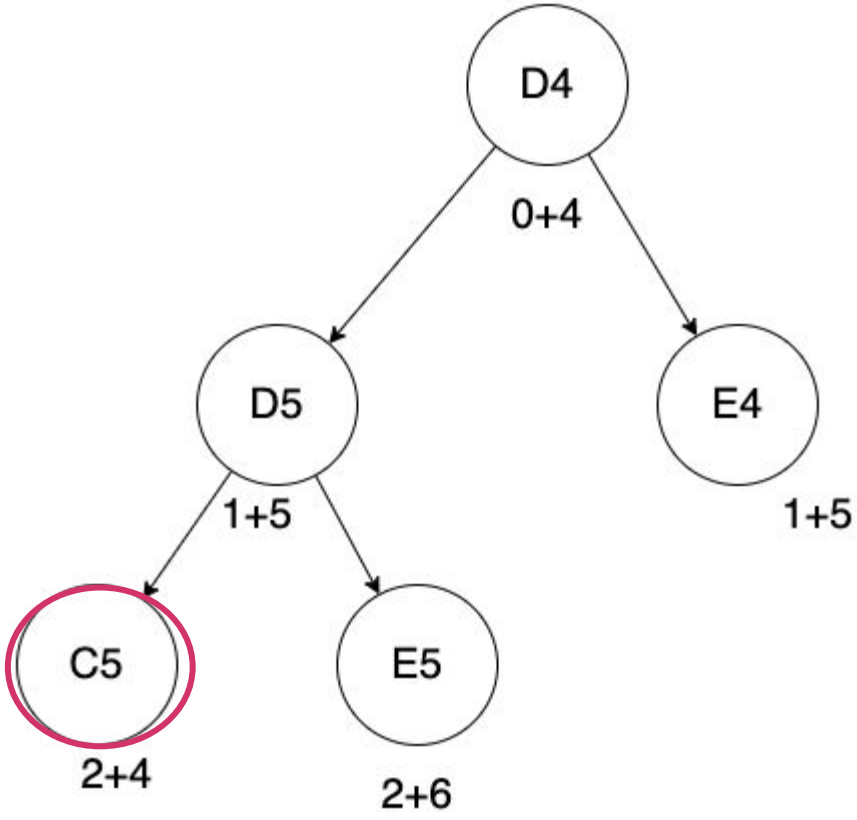
D5 = 1 + 5

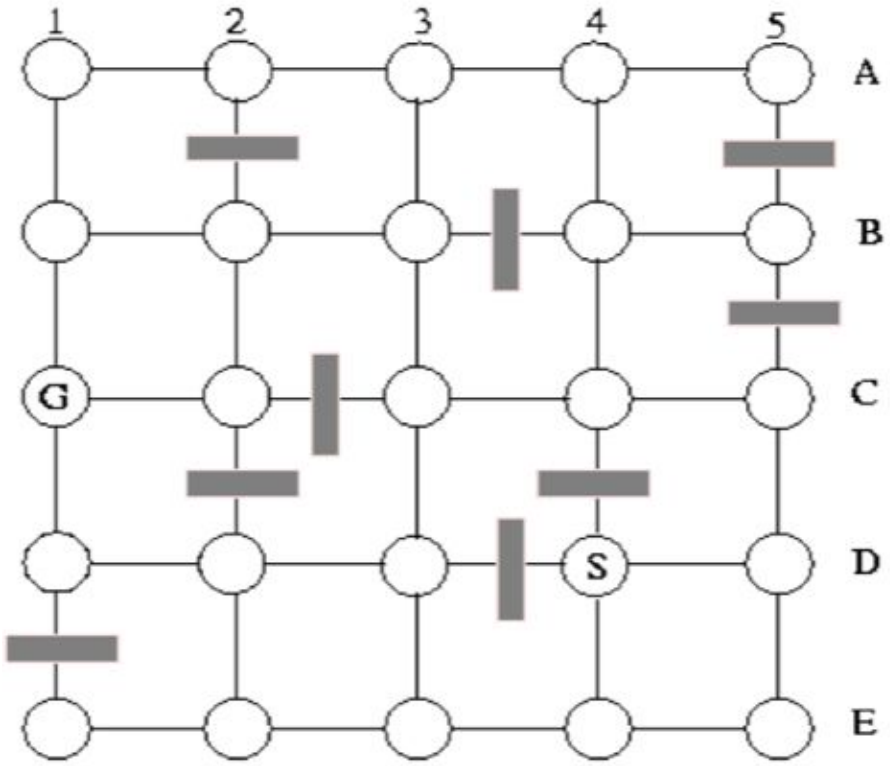


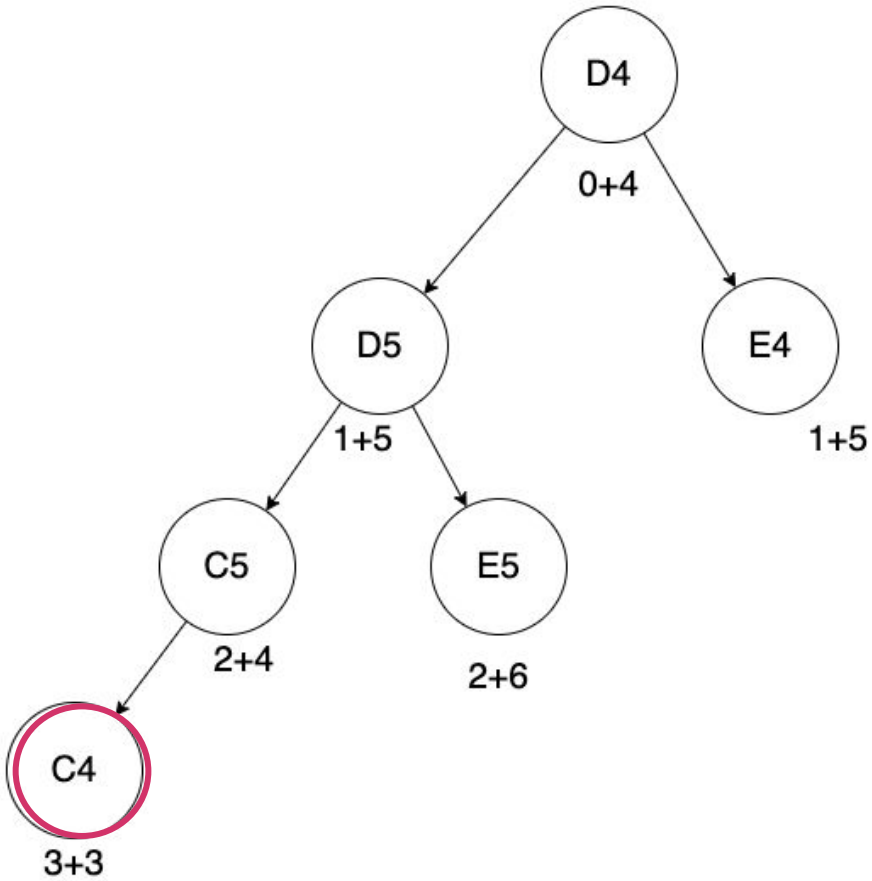

1
2
3
4
5
A
B
C
D
E
G
S

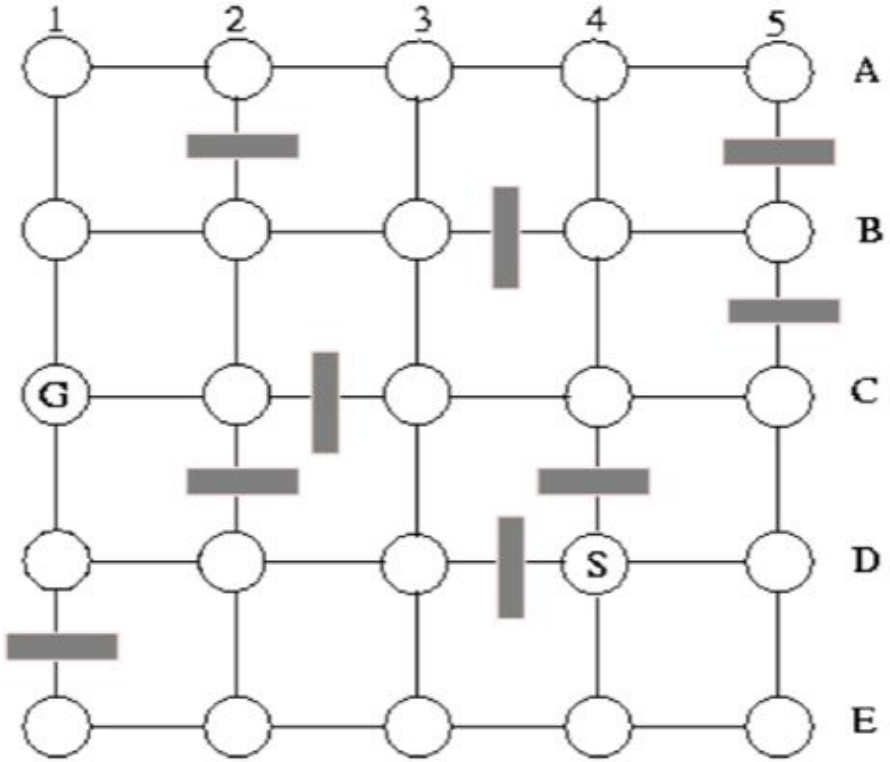

D4
0+4
D5
1+5
E4
1+5

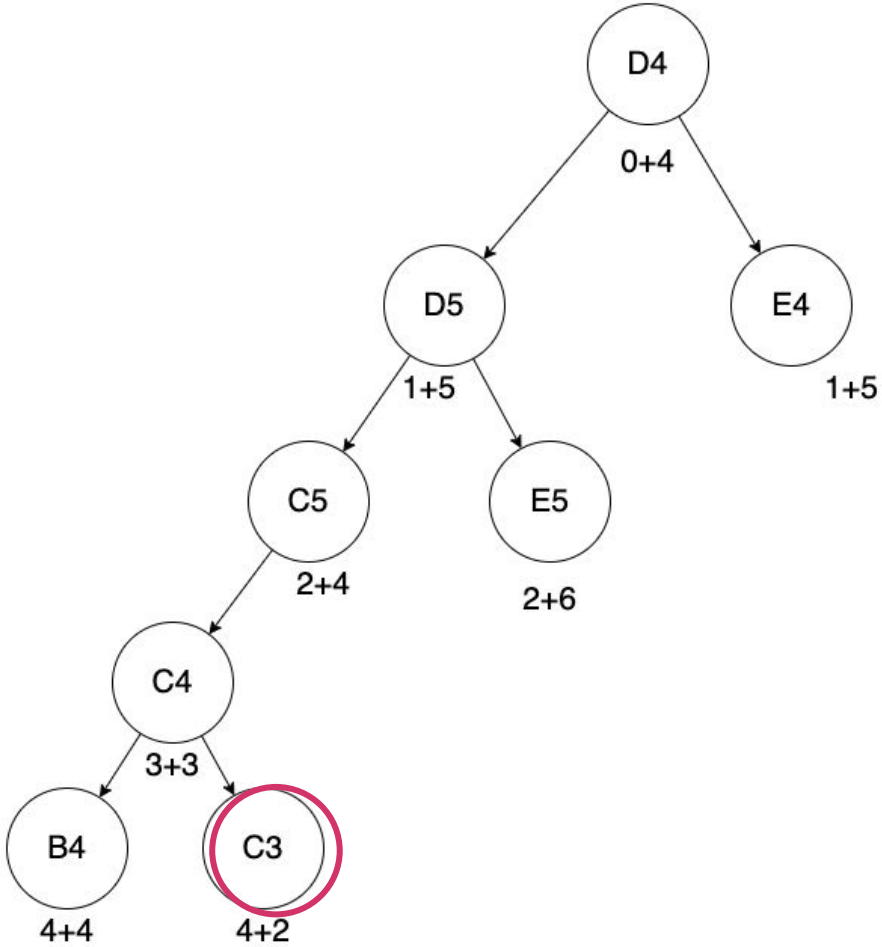

1
2
3
4
5
A
B
C
D
E
G
S

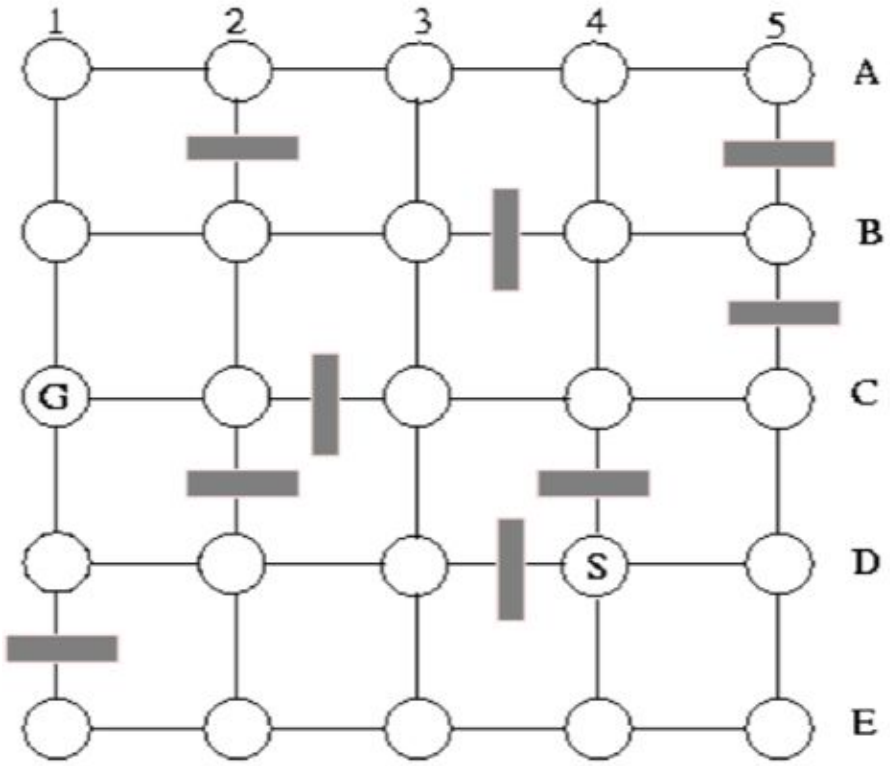

D4
0+4
D5
E4
1+5
1+5
C5
E5
2+4
2+6

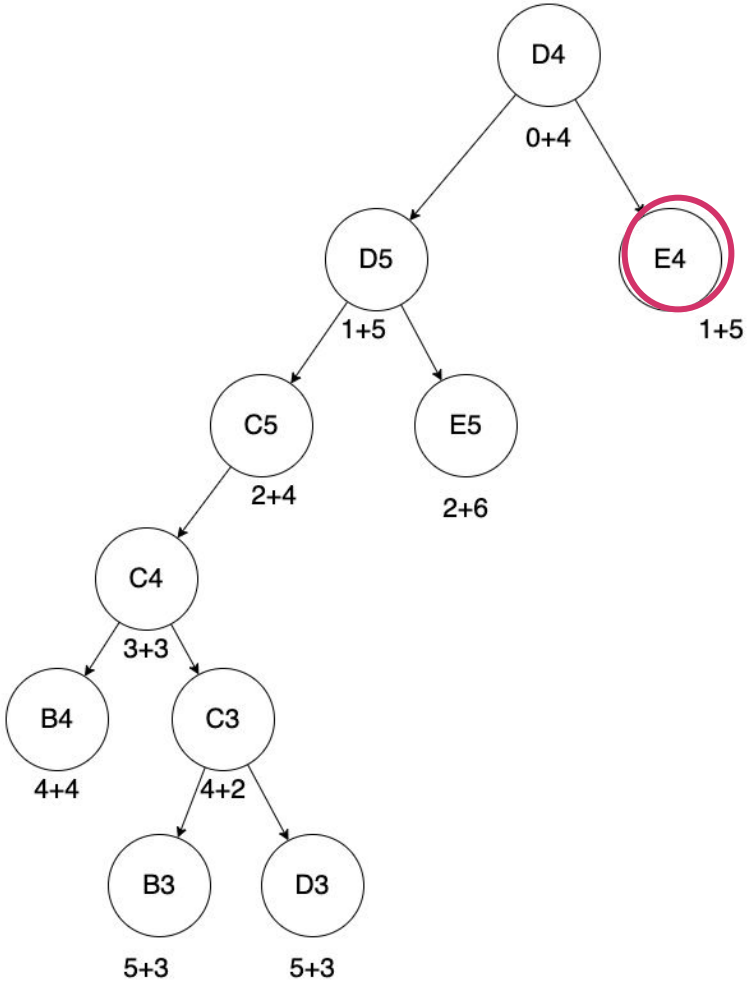

1
2
3
4
5
A
B
C
D
E
G
S

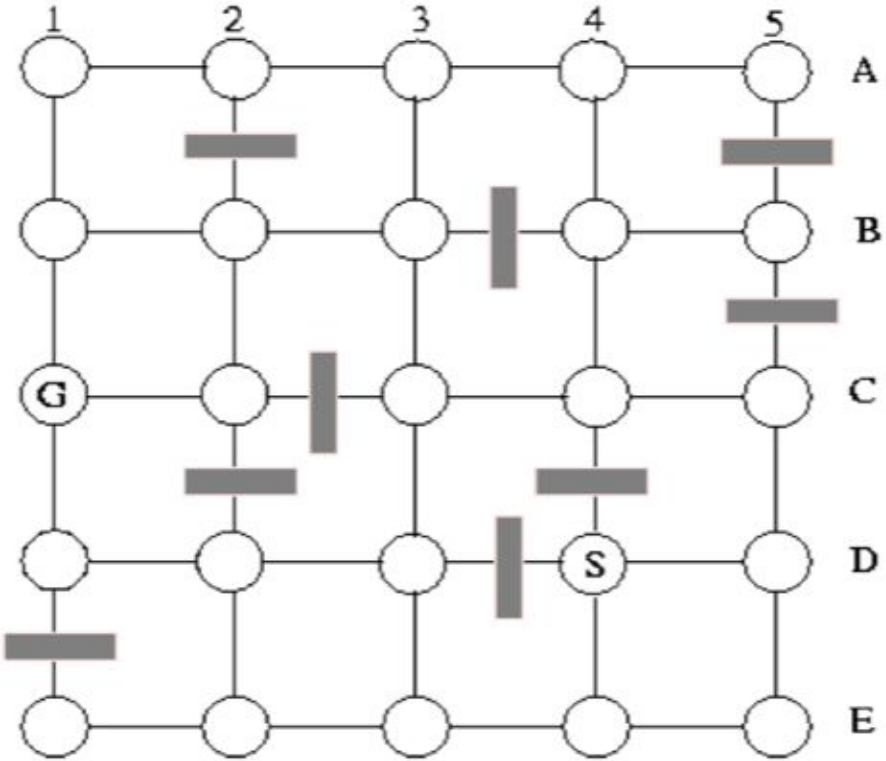

D4
0+4
D5
1+5
E4
1+5
C5
2+4
E5
2+6
C4
3+3

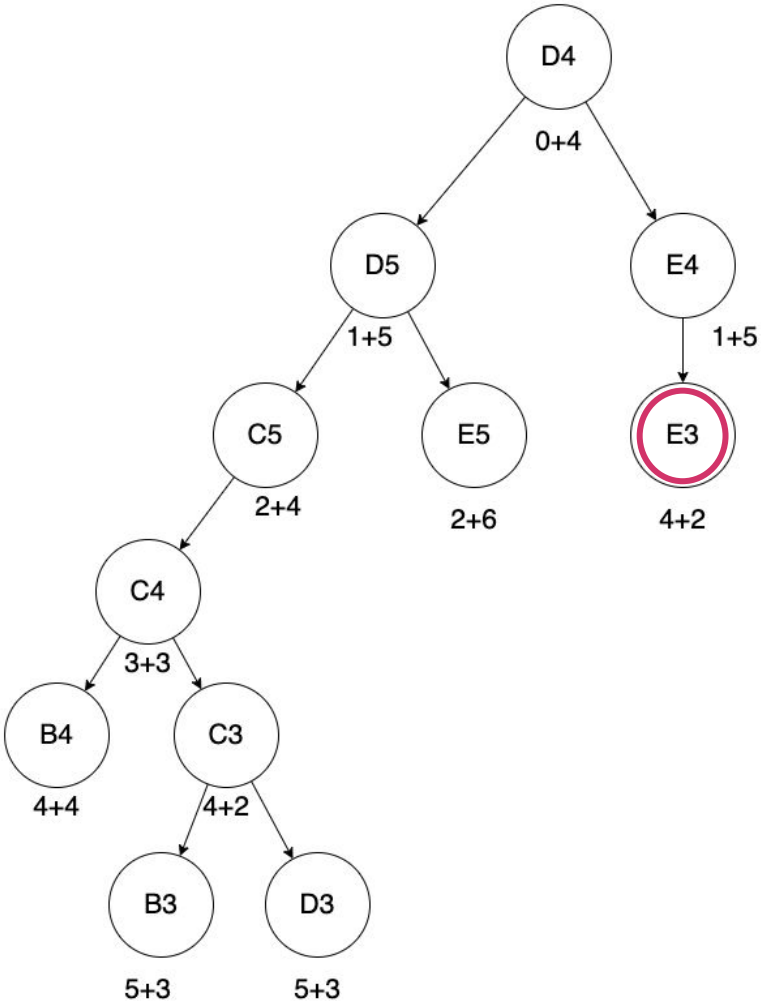

1
2
3
4
5
A
B
C
D
E
G
S

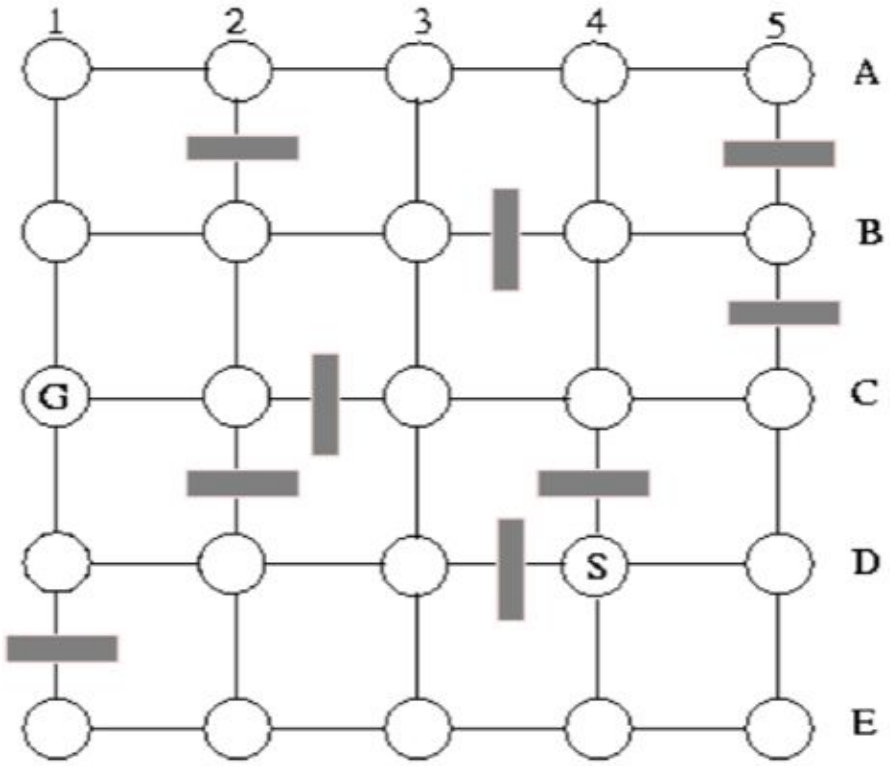

D4
0+4
D5
E4
1+5
1+5
C5
E5
2+4
2+6
C4
3+3
B4
C3
4+4
4+2

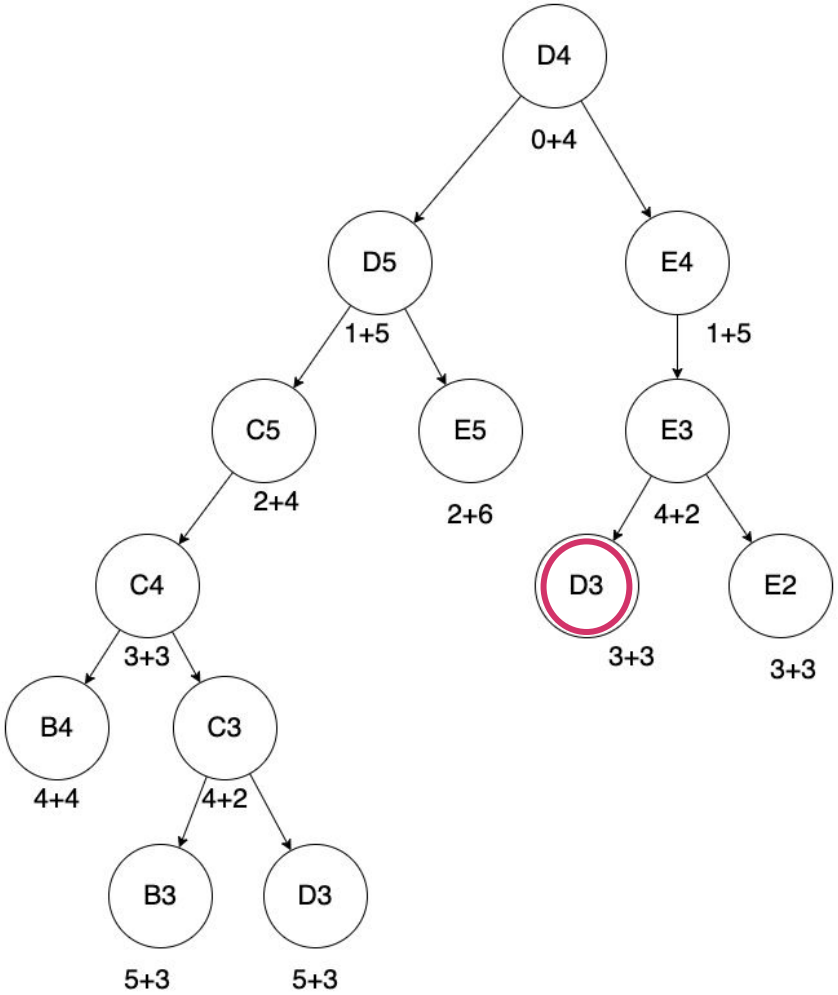

1
2
3
4
5
A
B
C
D
E
G
S

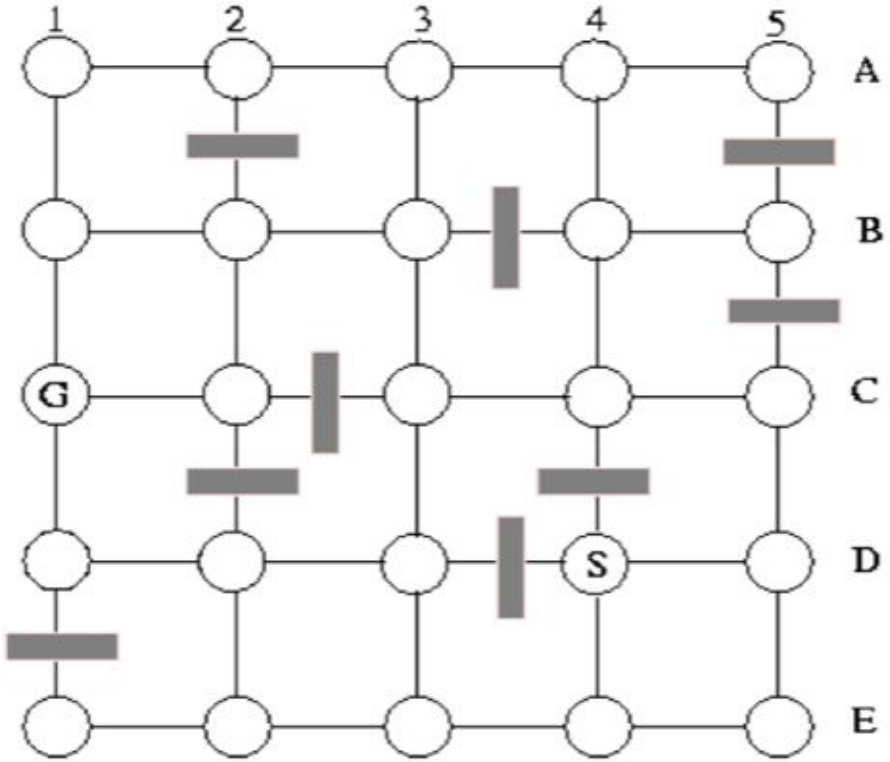

D4
0+4
D5
1+5
E4
1+5
C5
2+4
E5
2+6
C4
3+3
B4
4+4
C3
4+2
B3
5+3
D3
5+3

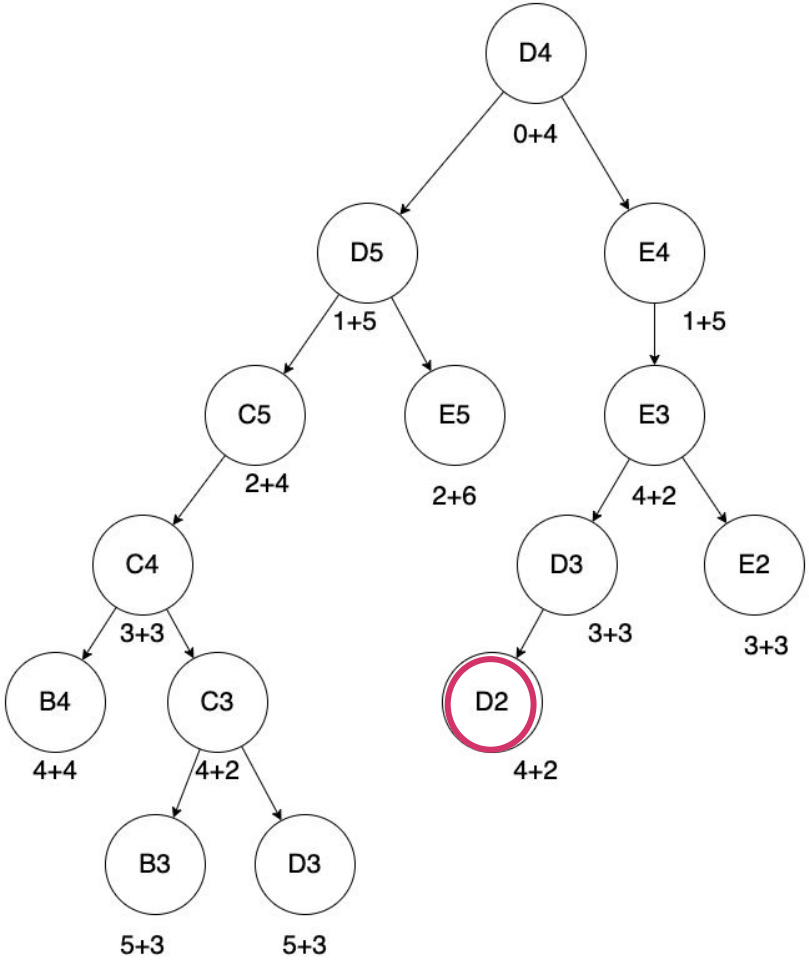

1
2
3
4
5
A
B
C
D
E
G
S

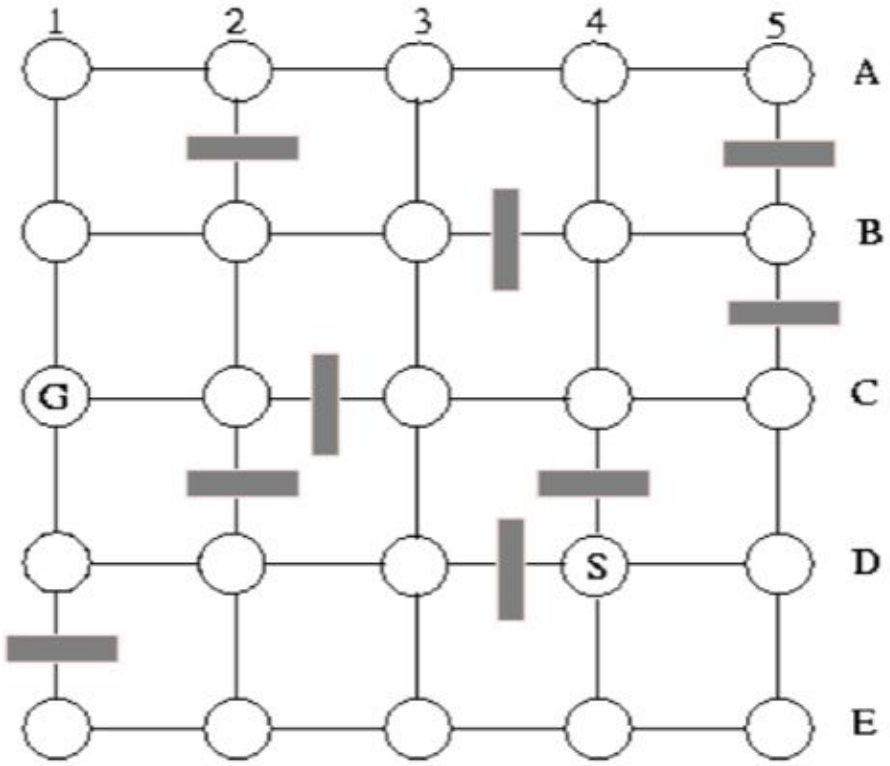

D4
0+4
D5
E4
1+5
1+5
C5
E5
E3
2+4
2+6
4+2
C4
3+3
B4
C3
4+4
4+2
B3
D3
5+3
5+3

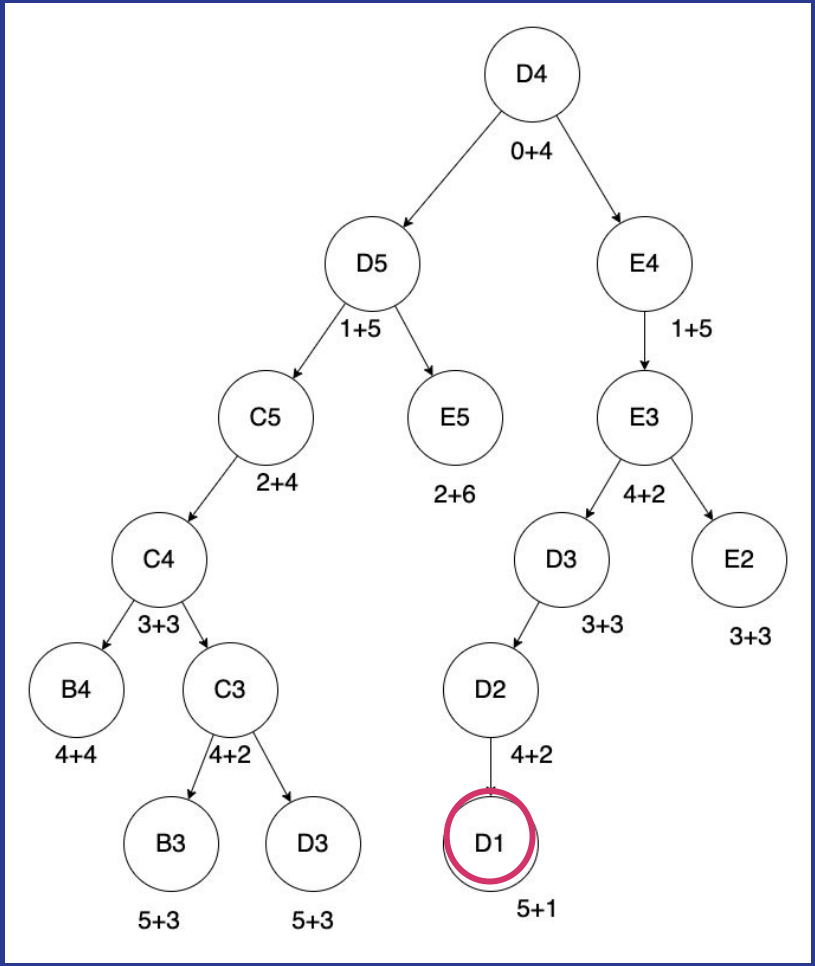

1
2
3
4
5
A
B
C
D
E
G
S

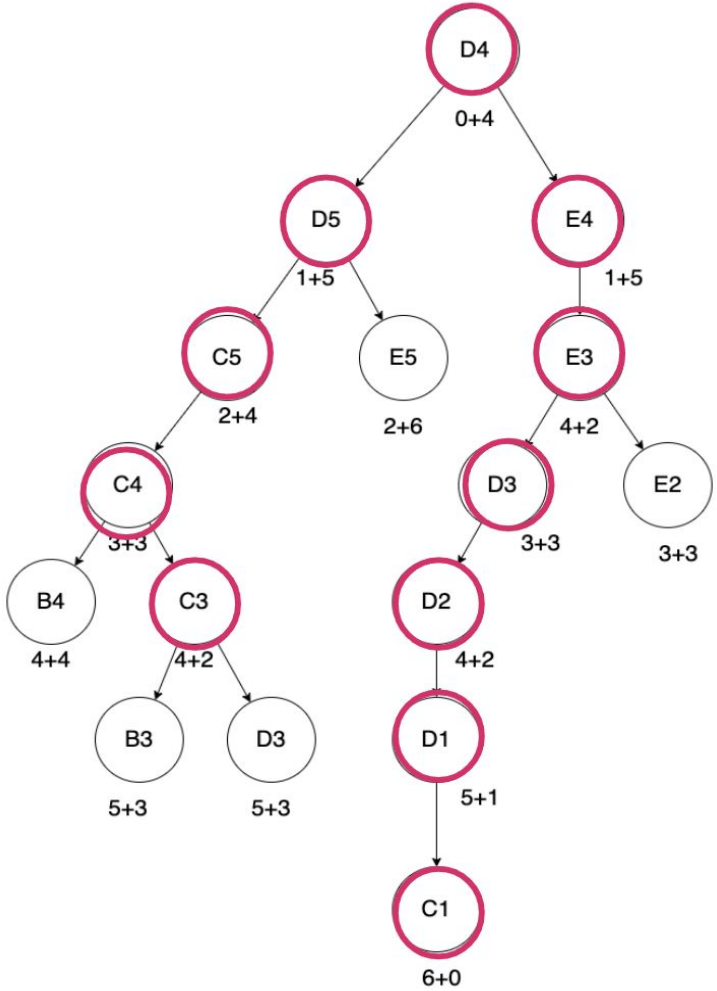

D4
0+4
D5
1+5
E4
1+5
C5
2+4
E5
2+6
E3
4+2
C4
3+3
D3
3+3
E2
3+3
B4
4+4
C3
4+2
B3
5+3
D3
5+3


1
2
3
4
5
A
B
C
D
E
G
S


D4
0+4
D5
E4
1+5
1+5
C5
E5
E3
2+4
2+6
4+2
C4
D3
E2
3+3
3+3
3+3
B4
C3
D2
4+4
4+2
4+2
B3
D3
5+3
5+3


1
2
3
4
5
A
B
C
D
E
G
S

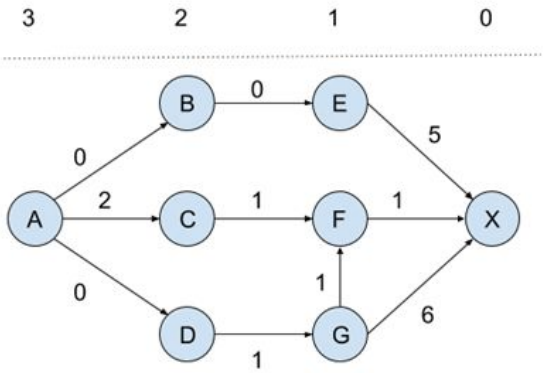

D4
0+4
D5
E4
1+5
1+5
C5
E5
E3
2+4
2+6
4+2
C4
D3
E2
3+3
3+3
3+3
B4
C3
D2
4+4
4+2
4+2
B3
D3
D1
5+3
5+3
5+1



Solution:

A*: D4 - D5 - C5 - C4 - C3 - E4 - E3 - D3 - D2 - D1 - C1

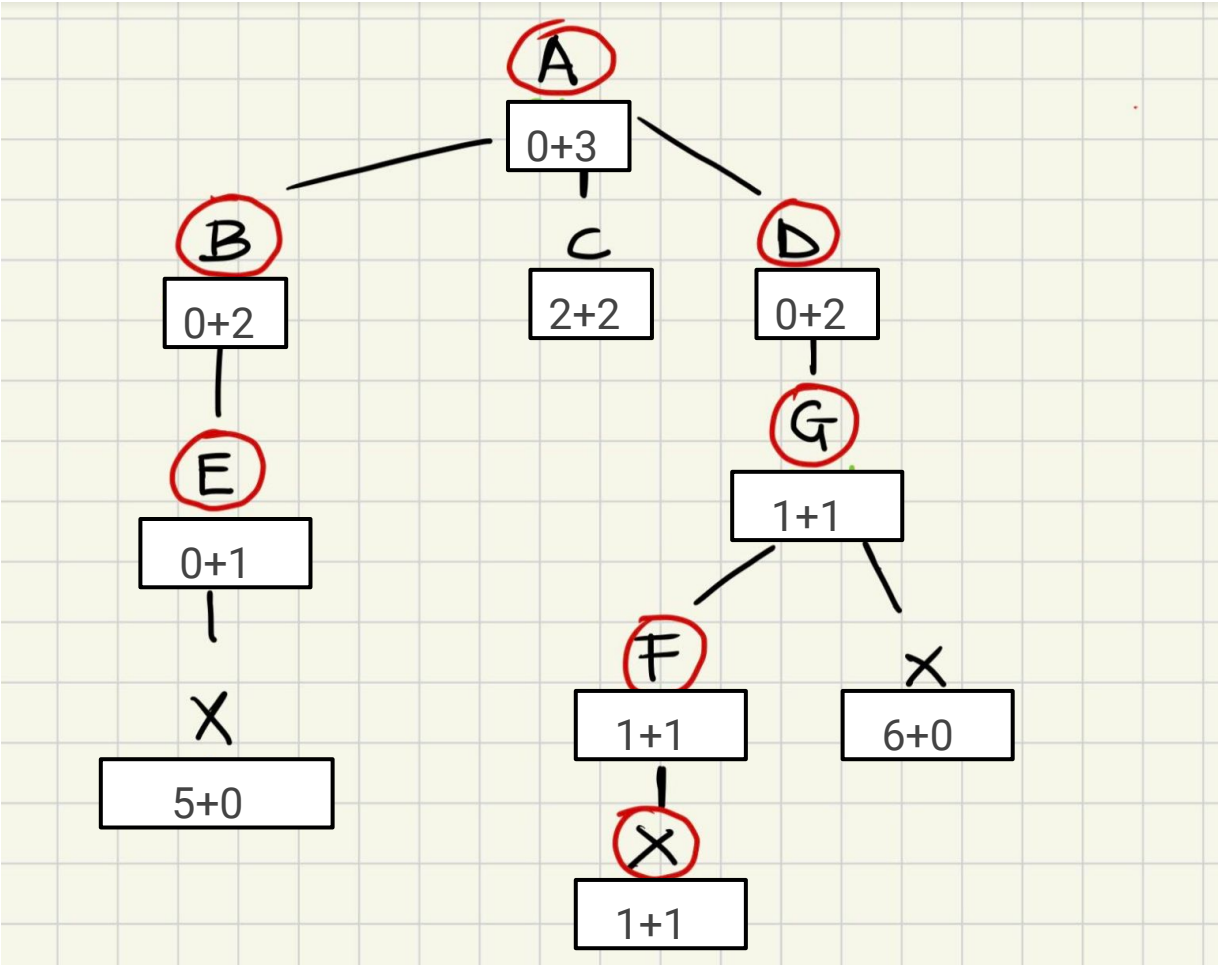
Exercise 6: Consider the following search problem:

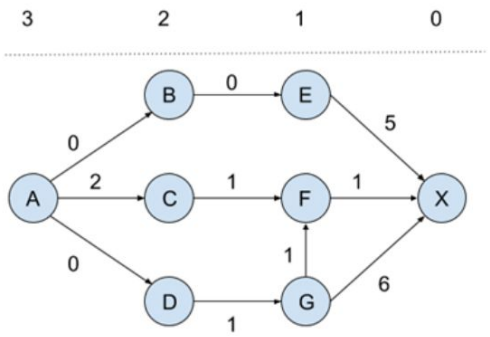


We start in A and want to reach X. The edge labels are the cost of moving from one node to another, and we want to find the path from A to X with the smallest total cost. Draw the A* search tree for this problem. The numbers above the dotted line represent the estimated cost to the goal state for all nodes below it (i.e. for nodes E, F and G, the estimated cost to the goal is 1, and the actual costs are 5, 1 and 6 respectively).

When two nodes have equal value, choose the one with the lowest letter alphabetically to expand first.

We say that a node is *added* when its evaluation function is computed. According to the algorithm from the lecture, only when a node is expanded it is checked whether it is a solution or not. A node is *expanded* when its children are added.

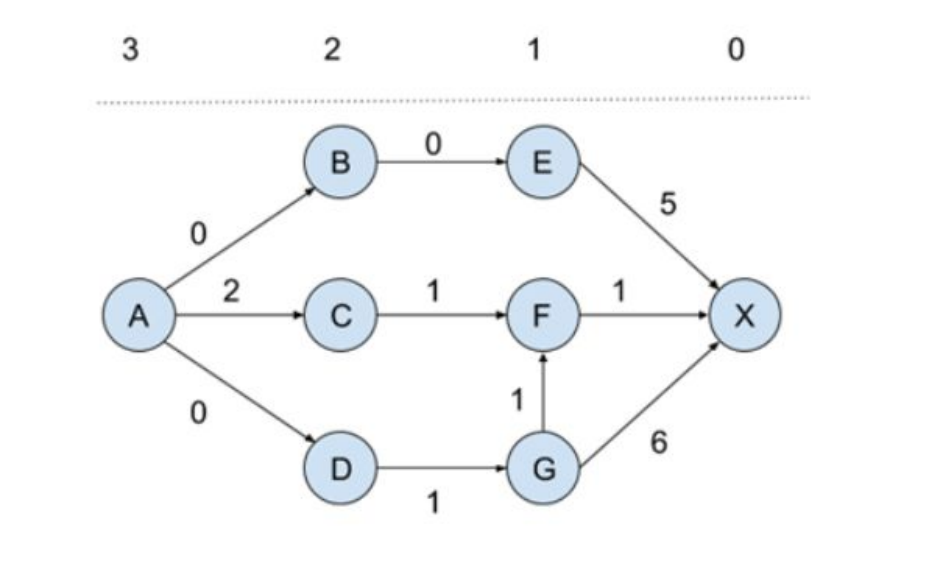




f(n) = g(n) + h(n)

h(n) = numbers on dotted lines
g(n) = actual costs

6.a Do the estimates above the dotted line lead to an admissible evaluation function (answer yes/no)?



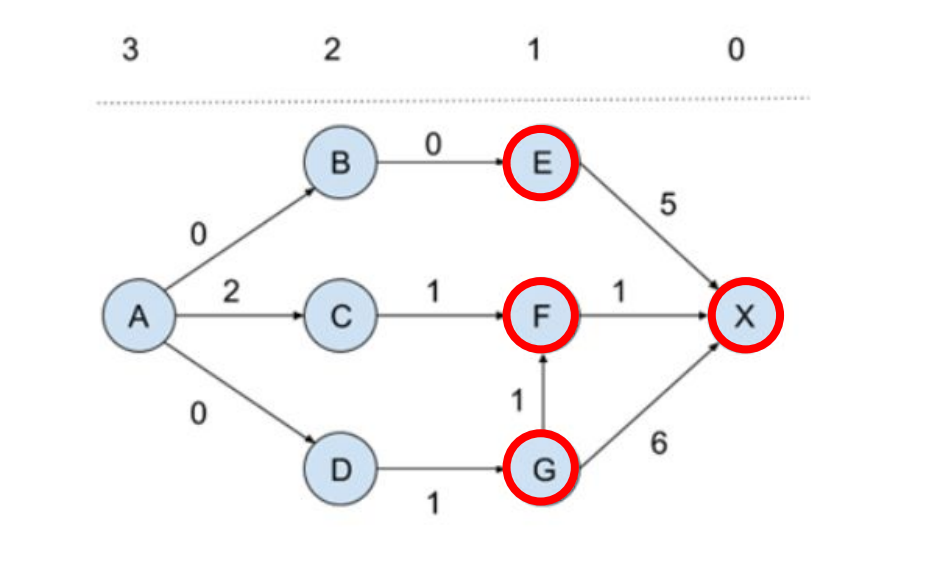
- The heuristic is admissible if it never overestimates the actual cost to reach the goal.
- Our case: 3 is the max estimate we have
- We cannot (actually) reach the goal in less than 3 steps
- Then the heuristic is admissible

6.a Solution

Do the estimates above the dotted line lead to an admissible evaluation function (answer yes/no)? YES, there is no way you can get from A to X than in 3 steps, and so on for all the other nodes.

6b: How often is the goal state added?

We say that a node is *added* when its evaluation function is computed. According to the algorithm from the lecture, only when a node is expanded it is checked whether it is a solution or not. A node is *expanded* when its children are added.

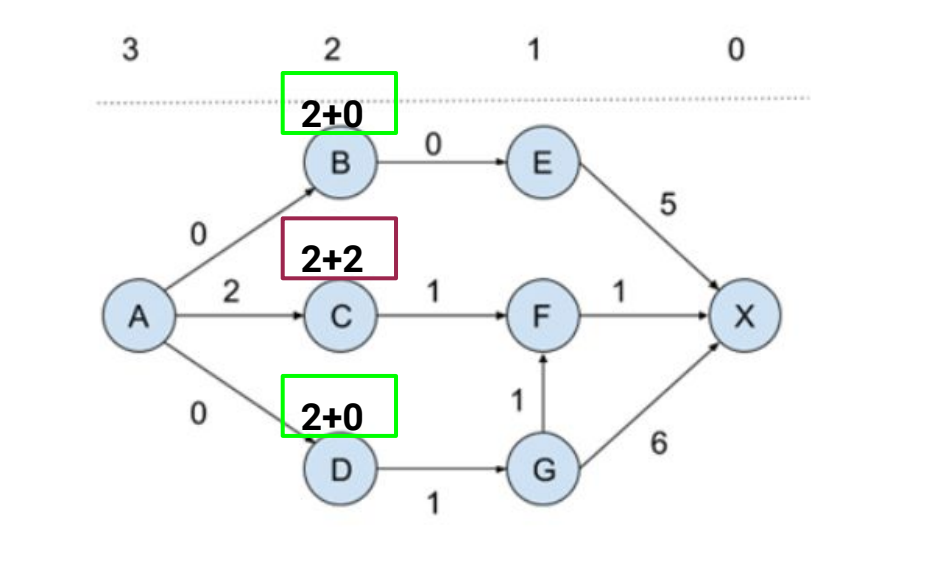


6.b Solution

How often is the goal state added? 3 times, when you explore E, G and F

6c: Which node is never expanded?

We say that a node is *added* when its evaluation function is computed. According to the algorithm from the lecture, only when a node is expanded it is checked whether it is a solution or not. A node is *expanded* when its children are added.



6.c Solution

- Which node is never expanded? C

ADVERSARIAL SEARCH/GAMES

```
function MINIMAX-DECISION(state) returns an action
  inputs: state, current state in game
  v←MAX-VALUE(state)
  return the action in SUCCESSORS(state) with value v
```

```
function MAX-VALUE(state) returns a utility value
  if TERMINAL-TEST(state) then return UTILITY(state)
  v ← - ∞
  for a,s in SUCCESSORS(state) do
    v ← MAX(v,MIN-VALUE(s))
  return v
```

```
function MIN-VALUE(state) returns a utility value
  if TERMINAL-TEST(state) then return UTILITY(state)
  v ← ∞
  for a,s in SUCCESSORS(state) do
    v ← MIN(v,MAX-VALUE(s))
  return v
```

```
function MINIMAX-DECISION(state) returns an action
  inputs: state, current state in game
  v ← MAX-VALUE(state)
  return the action in SUCCESSORS(state) with value v
```

- Input: `state`, which is the current state of the game.
- Process: This function initiates the Minimax algorithm by calling the `MAX-VALUE` function on the current `state`.
- Output: It returns an `action` that corresponds to the highest value returned by `MAX-VALUE`. This action is what the algorithm has determined to be the best move for the maximizing player

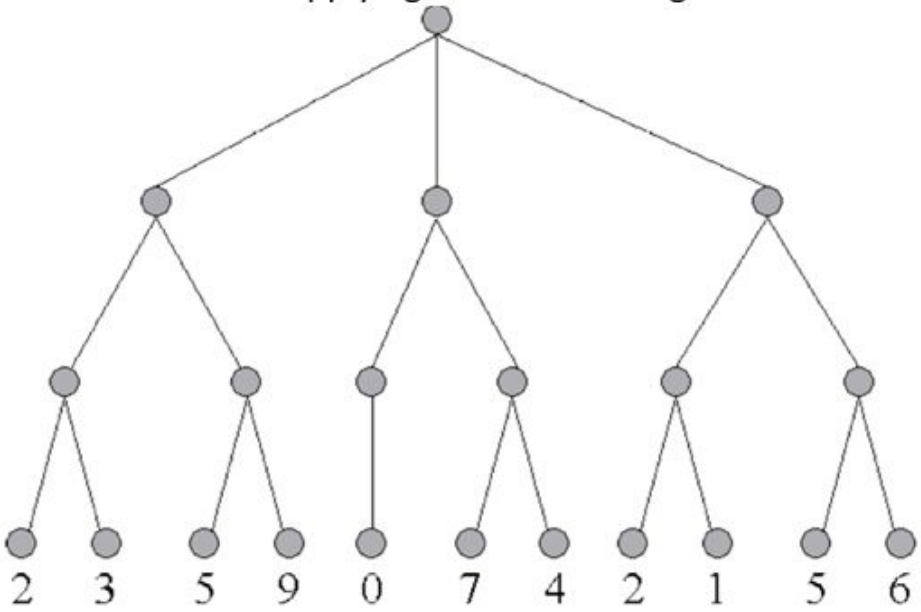
```
function MAX-VALUE(state) returns a utility value
  if TERMINAL-TEST(state) then return UTILITY(state)
  v ← - ∞
  for a,s in SUCCESSORS(state) do
    v ← MAX(v,MIN-VALUE(s))
  return v
```

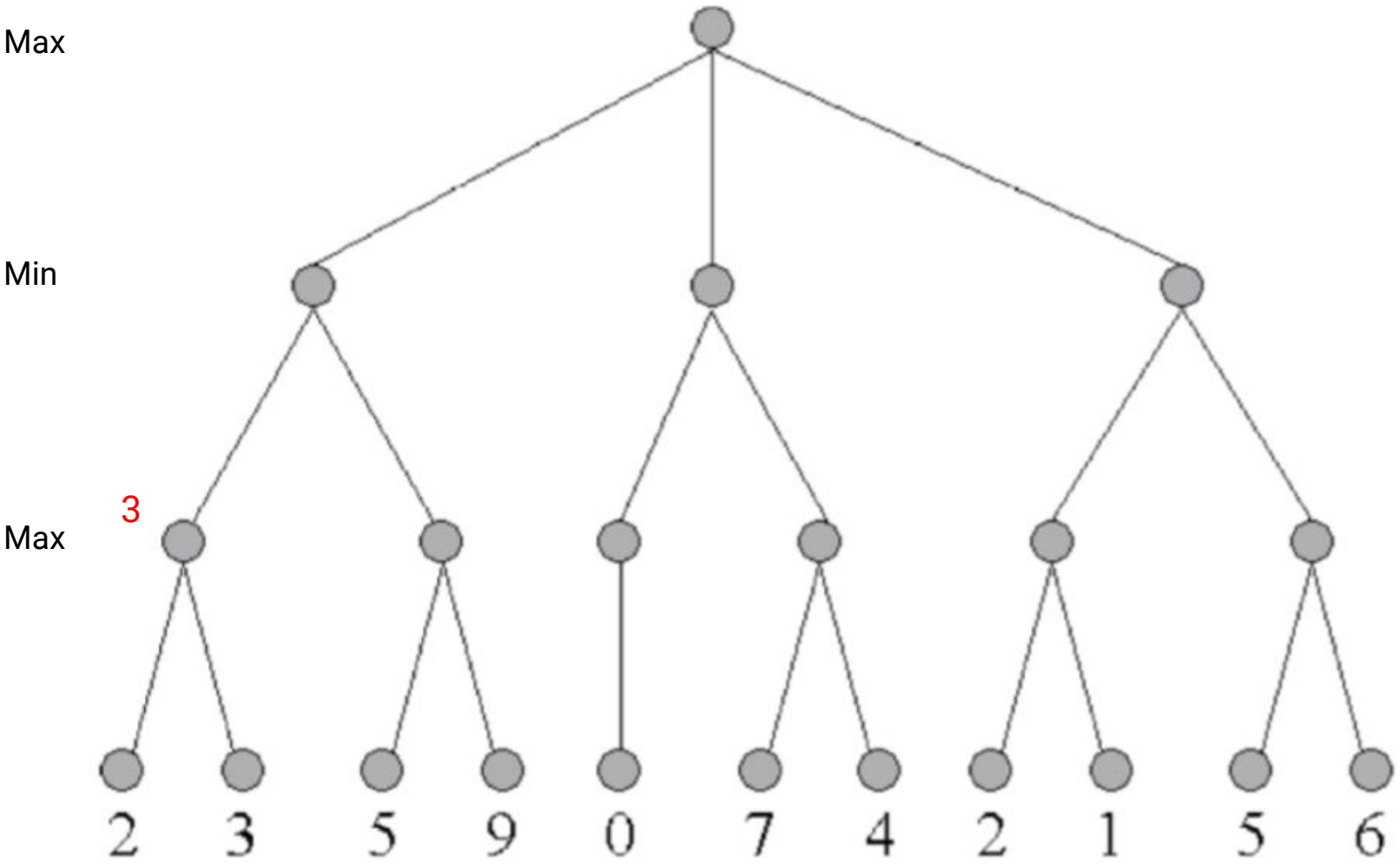
- Input: `state`, which represents a specific configuration of the game at a given time.
- Process: This function is called recursively and does the following:
  a. Checks if the `state` is a terminal state (i.e., a state where the game is over) using `TERMINAL-TEST`. If it is, it returns the `UTILITY` of the state, which is a measure of how good that state is for the maximizing player.
  b. If not a terminal state, it initializes `v` to negative infinity to find the maximum value as it evaluates further states.
  c. It goes through all possible `SUCCESSORS` of the current `state` (i.e., all valid moves that can be made from the current state) and calls `MIN-VALUE` on each, attempting to find the move with the highest utility.
- Output: Returns the highest value `v` found among all possible successor states.

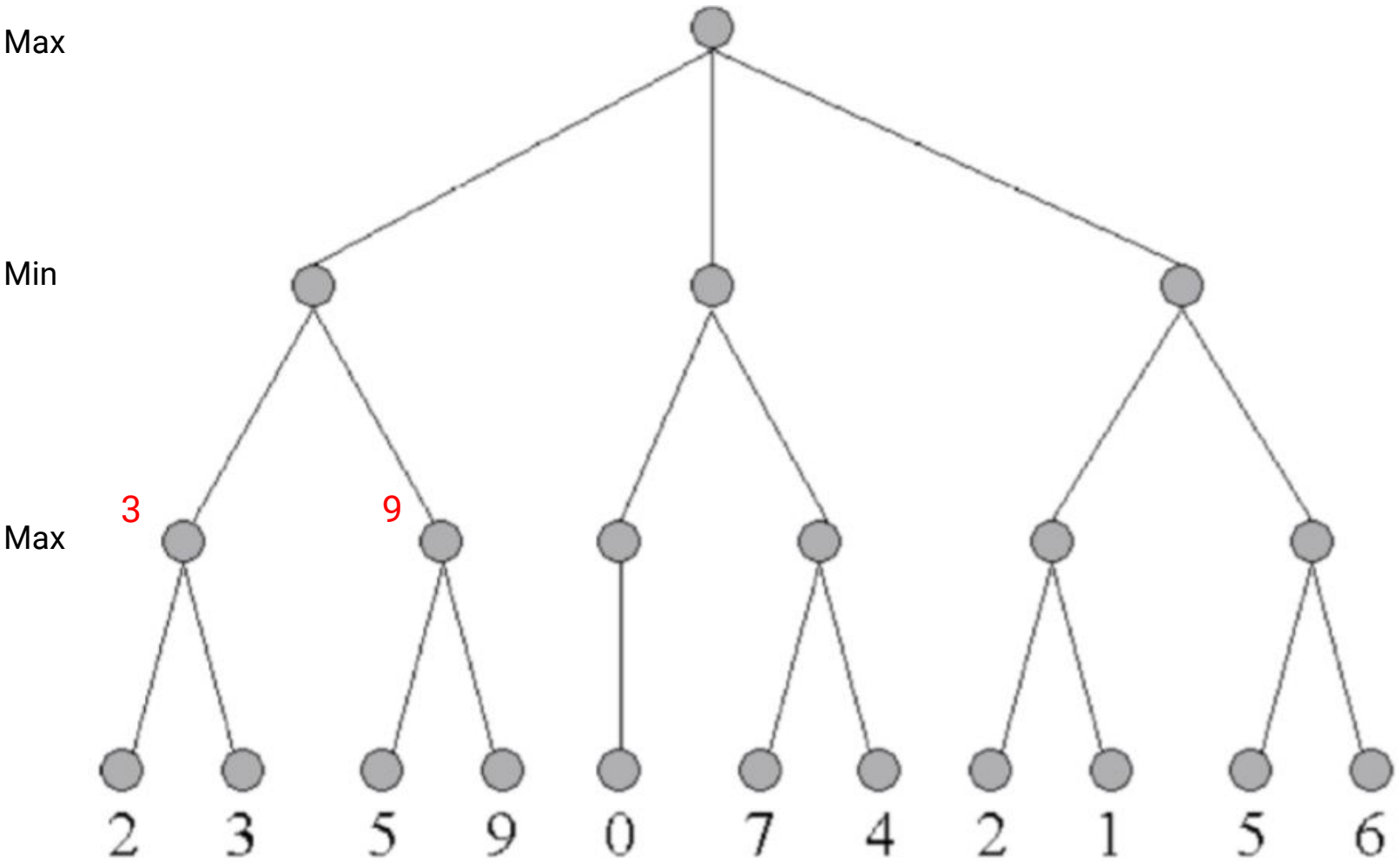
```
function MIN-VALUE(state) returns a utility value
  if TERMINAL-TEST(state) then return UTILITY(state)
  v ← ∞
  for a,s in SUCCESSORS(state) do
    v ← MIN(v,MAX-VALUE(s))
  return v
```

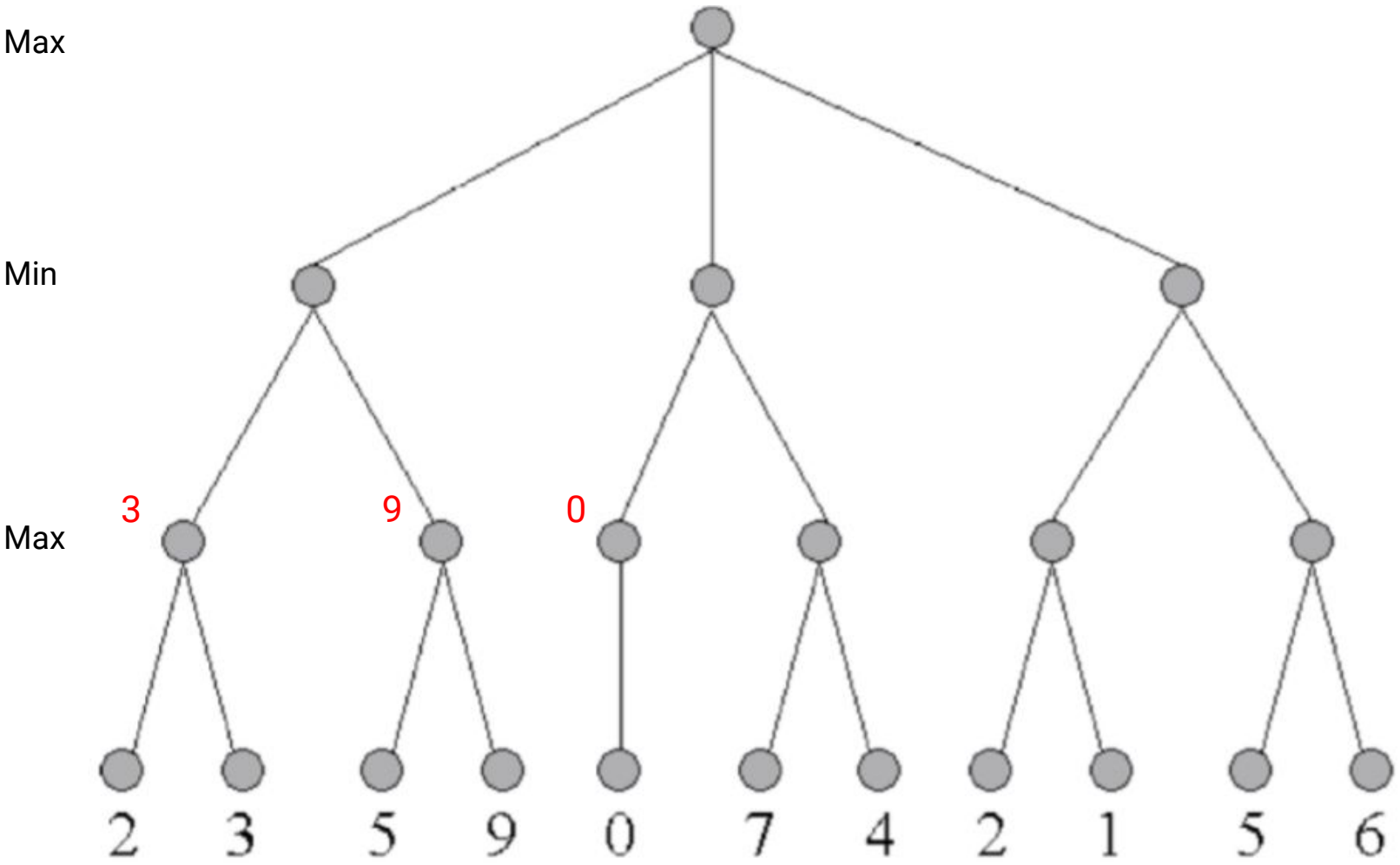
- Input: `state`: The current node in the search tree.
- Process: Use `TERMINAL-TEST` to see if this node is a goal. If it's a goal, get its value with `UTILITY`. If not a goal, start with a very high value (infinity) because we're looking for the lowest value. Look at each connected node (successors) and find out what the highest value (worst for us) the next layer of nodes could return. Of all these highest values from the next layer, pick the lowest one.
- Output: The lowest value found `v`, which represents the least favorable outcome at this level of the search tree.

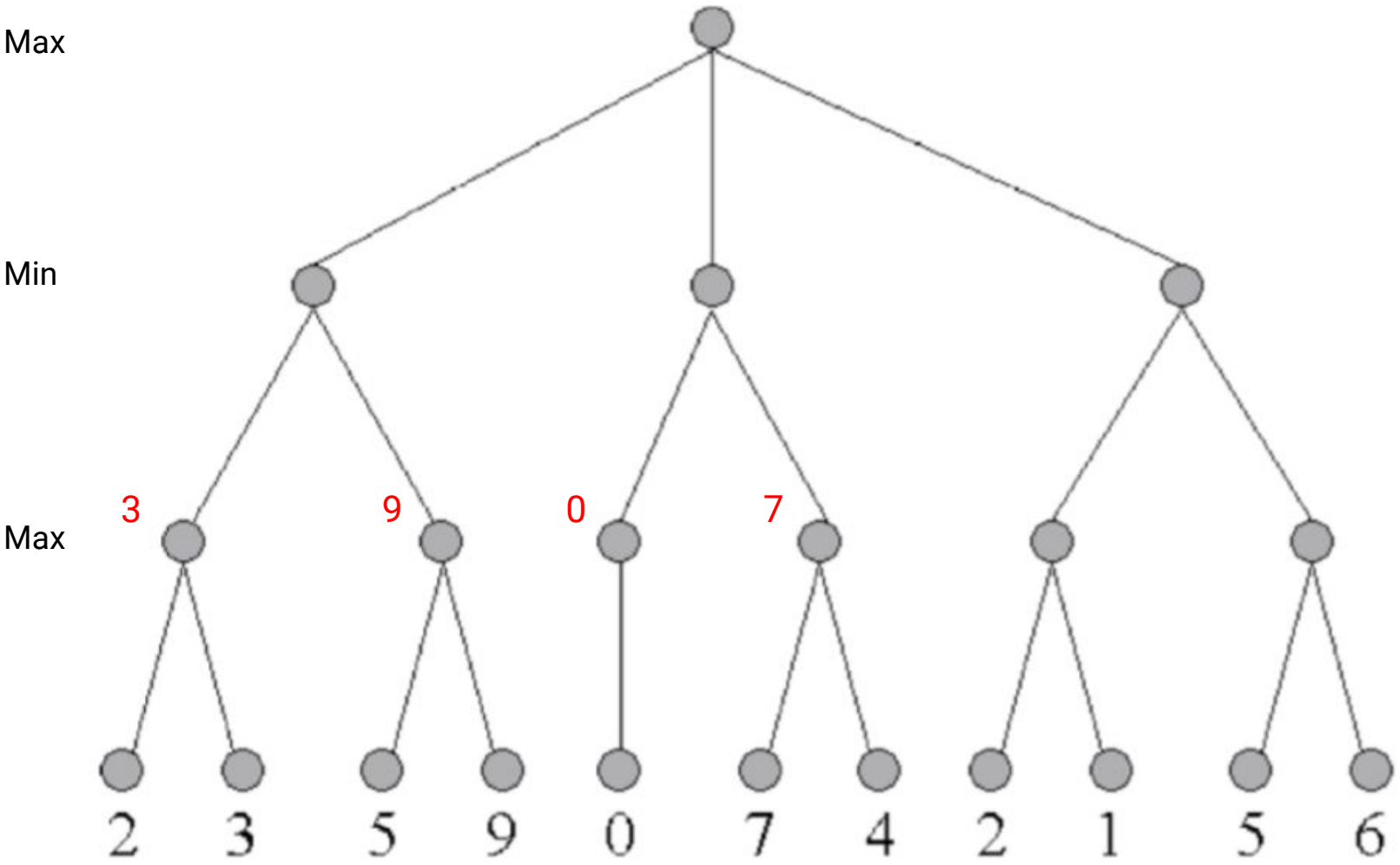
Exercise 7: Consider the following 2 player game tree. If MAX starts the game, what is the heuristic value after applying the Minimax algorithm?

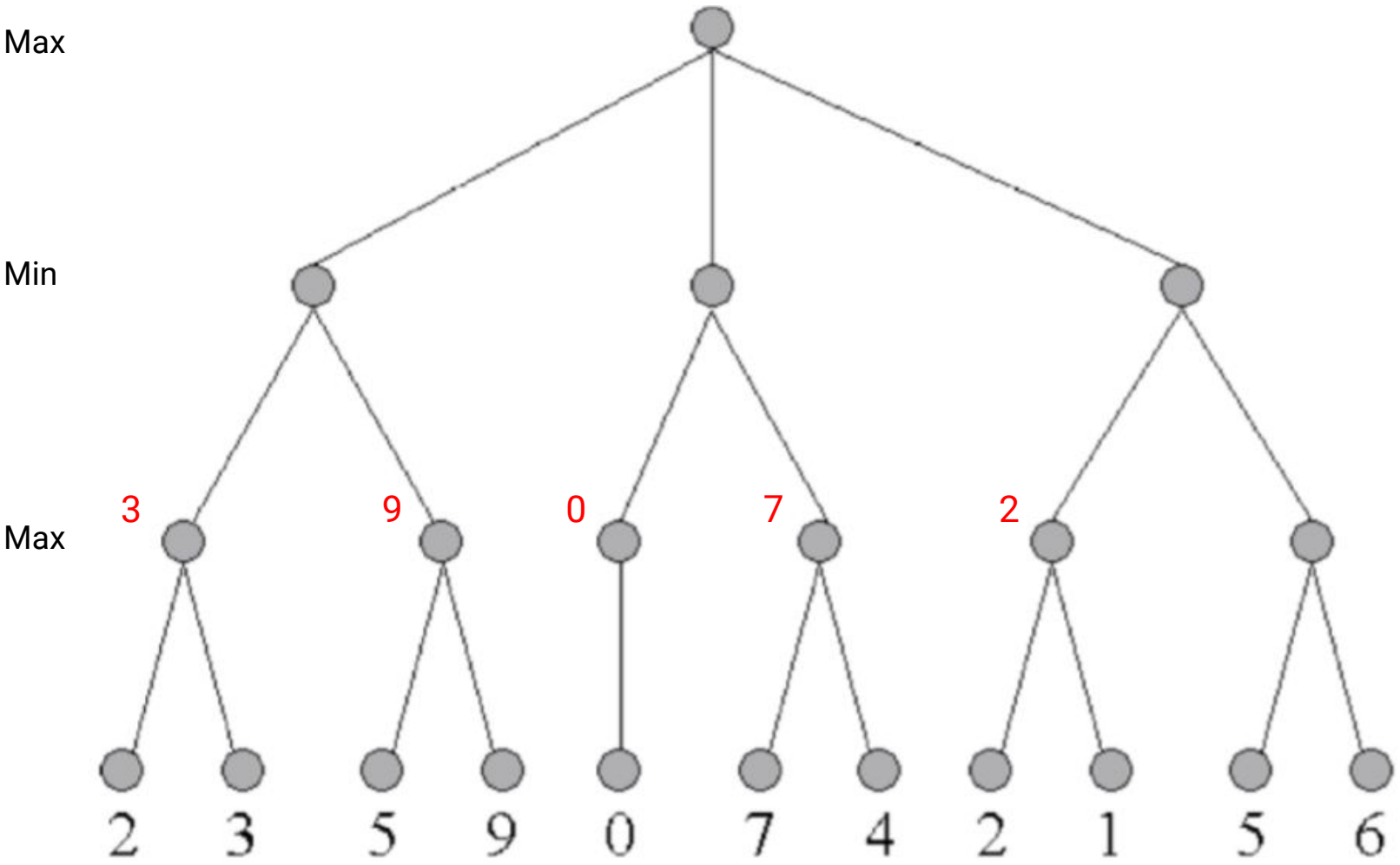


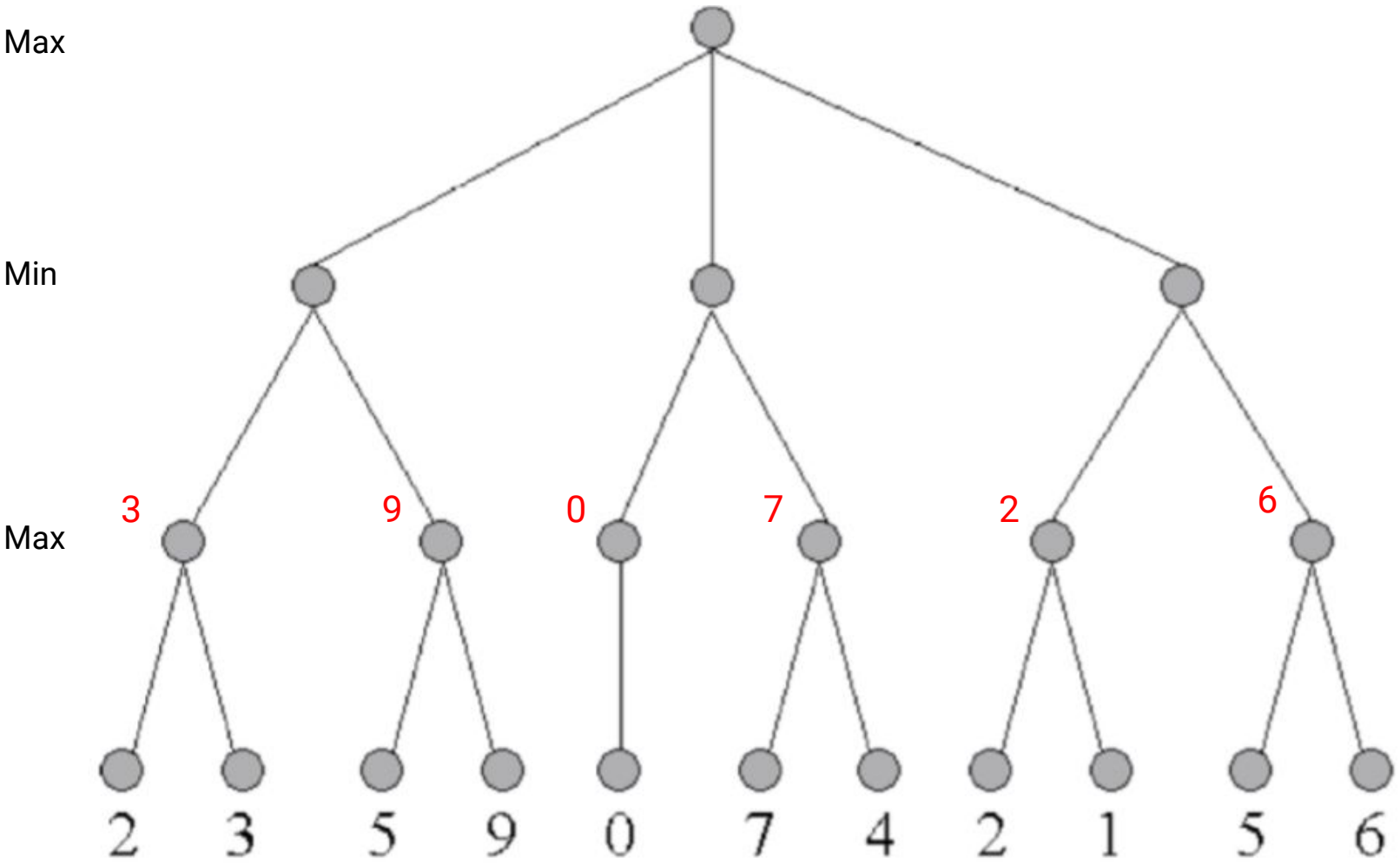

Max
Min
Max
3
2
3
5
9
0
7
4
2
1
5
6

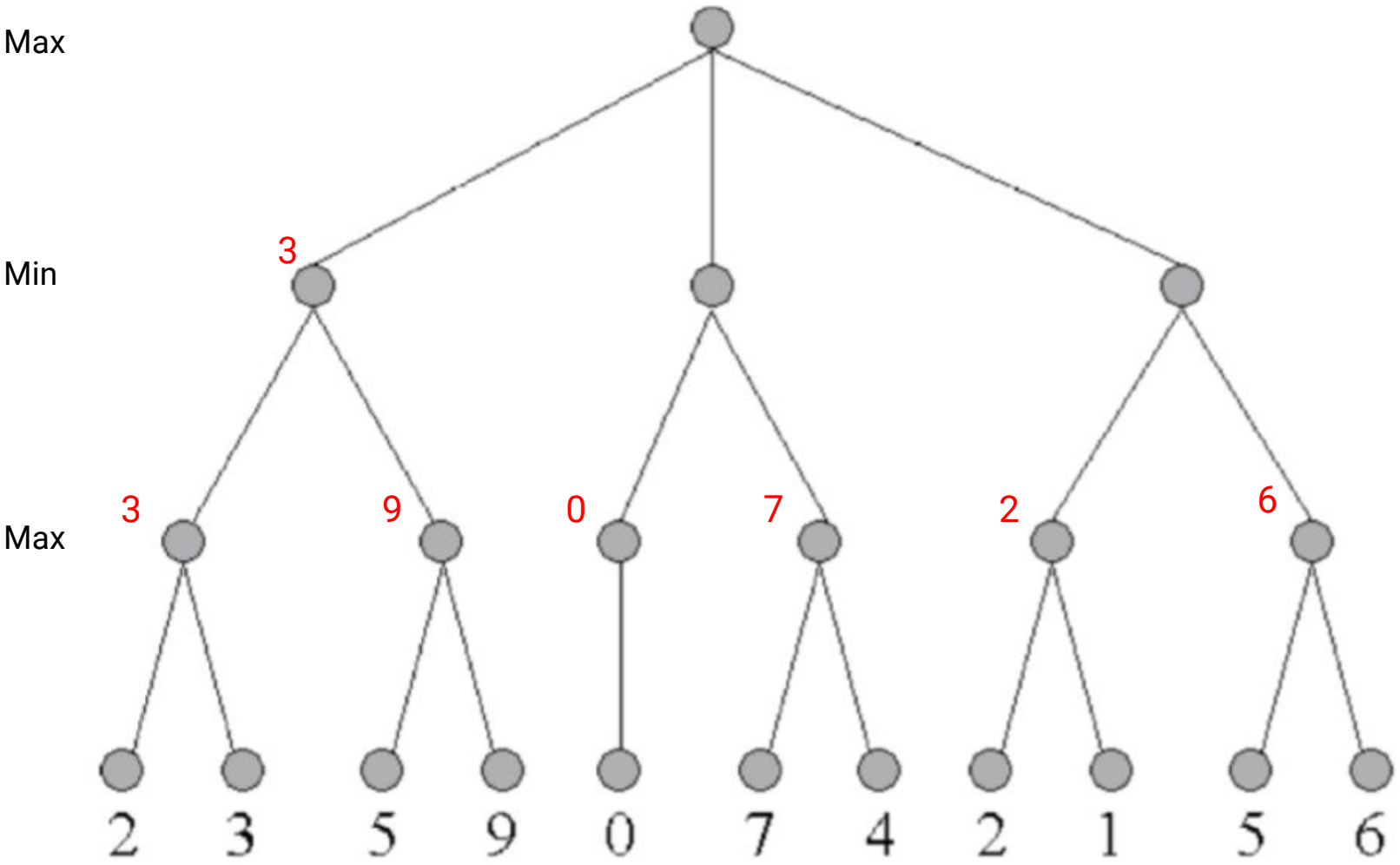

Max
Min
Max
3
9
2
3
5
9
0
7
4
2
1
5
6

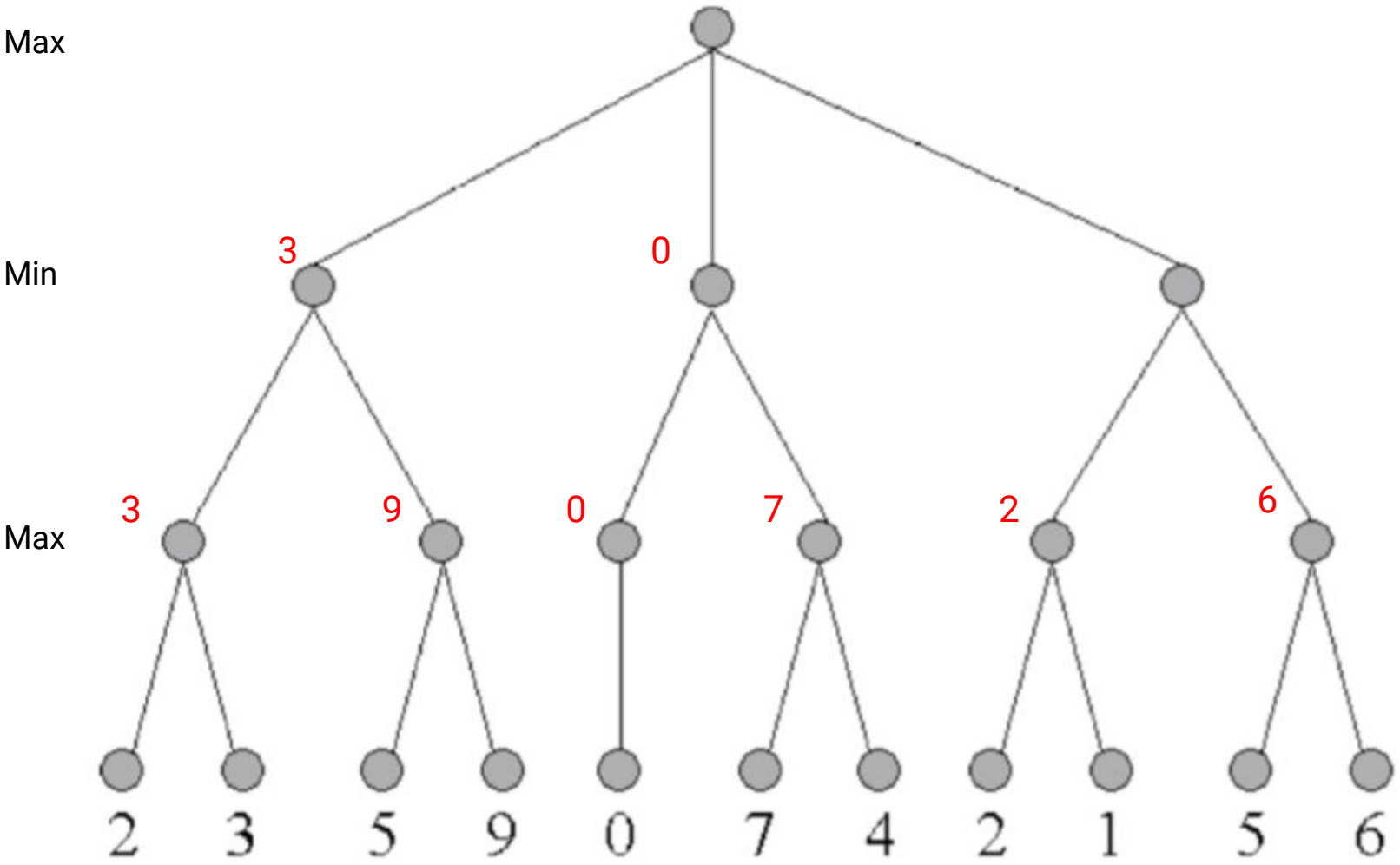

Max
Min
Max
3
9
0
2
3
5
9
0
7
4
2
1
5
6

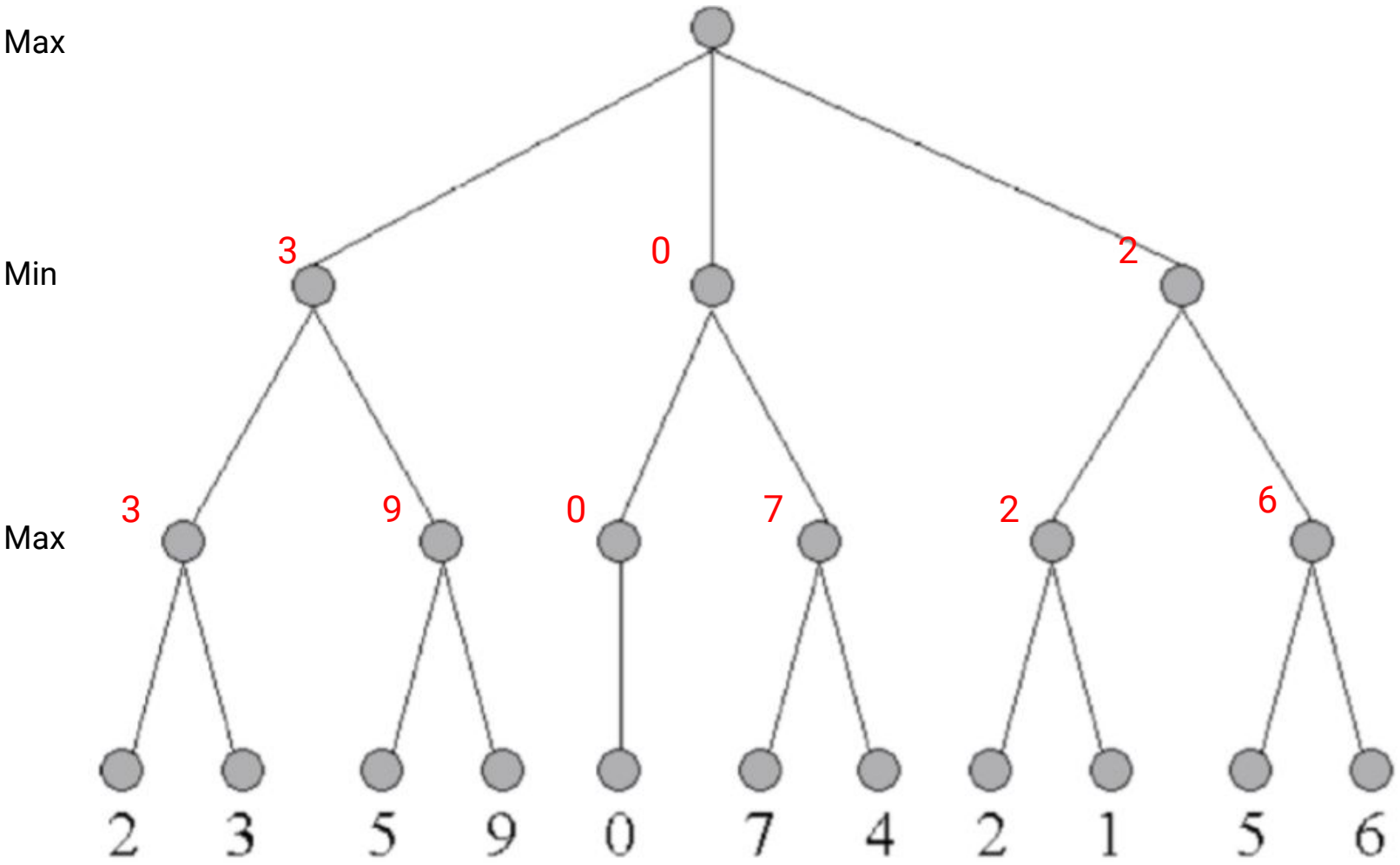

Max
Min
Max
3
9
0
7
2
3
5
9
0
7
4
2
1
5
6

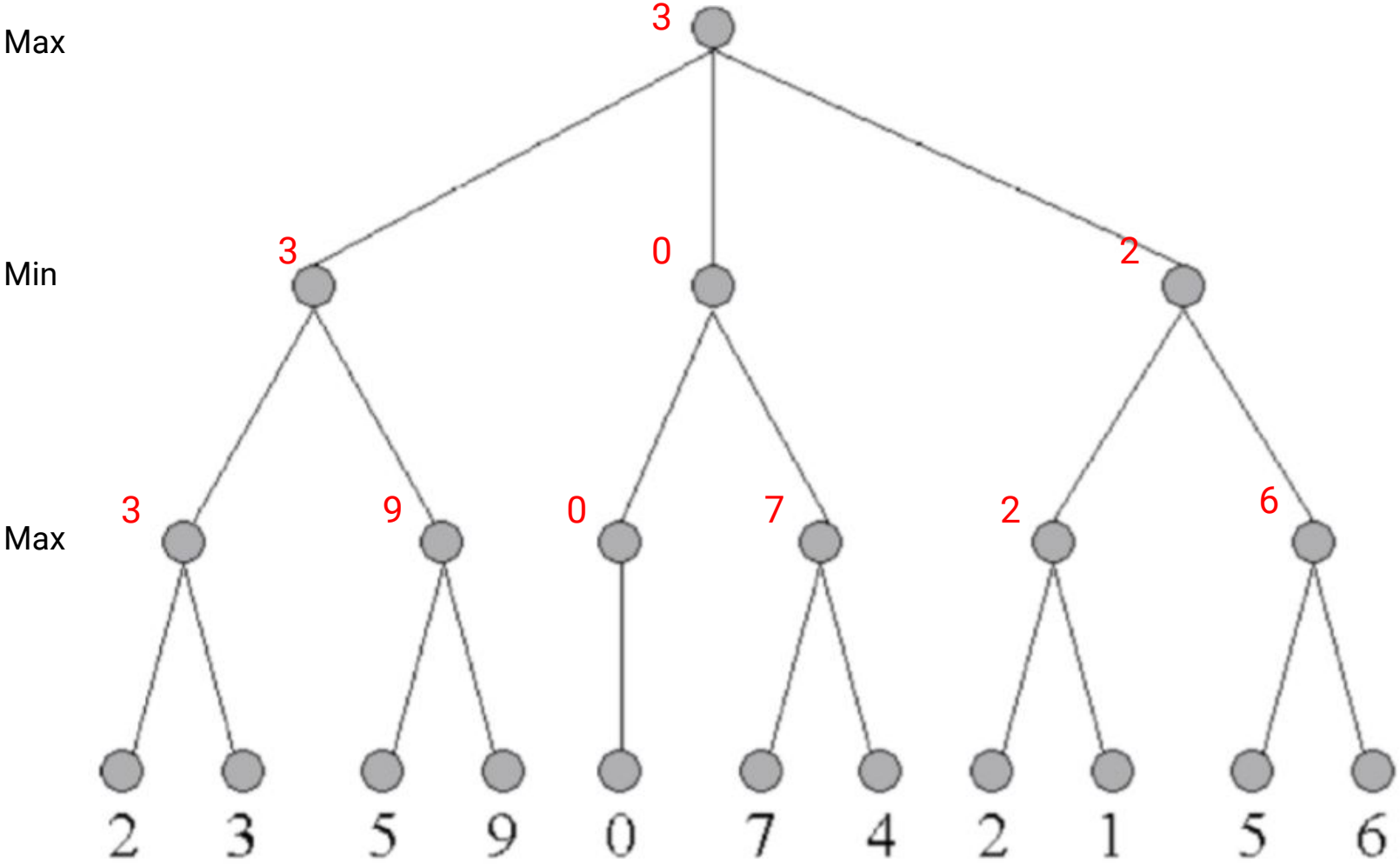

Max
Min
Max
3
9
0
7
2
2
3
5
9
0
7
4
2
1
5
6


Max
Min
Max
3
9
0
7
2
6
2
3
5
9
0
7
4
2
1
5
6


Max
Min
Max
3
3
9
0
7
2
6
2
3
5
9
0
7
4
2
1
5
6


Max
Min
Max
3
0
3
9
0
7
2
6
2
3
5
9
0
7
4
2
1
5
6


Max
Min
Max
3
0
2
3
9
0
7
2
6
2
3
5
9
0
7
4
2
1
5
6


Max
Min
Max
3
3
0
2
3
9
0
7
2
6
2
3
5
9
0
7
4
2
1
5
6

What is PRUNING?

As you evaluate the tree, if at any point the current node's value is such that the player can already find a better value (higher for maximizer or lower for minimizer), then further exploration of siblings is unnecessary, and these nodes are "pruned" or cut off from consideration.

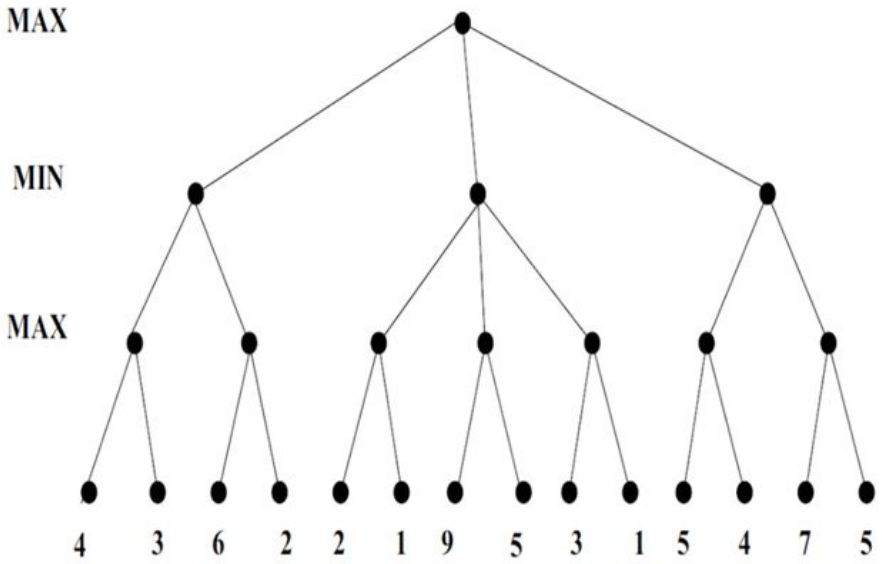
ALPHA-BETA PRUNING

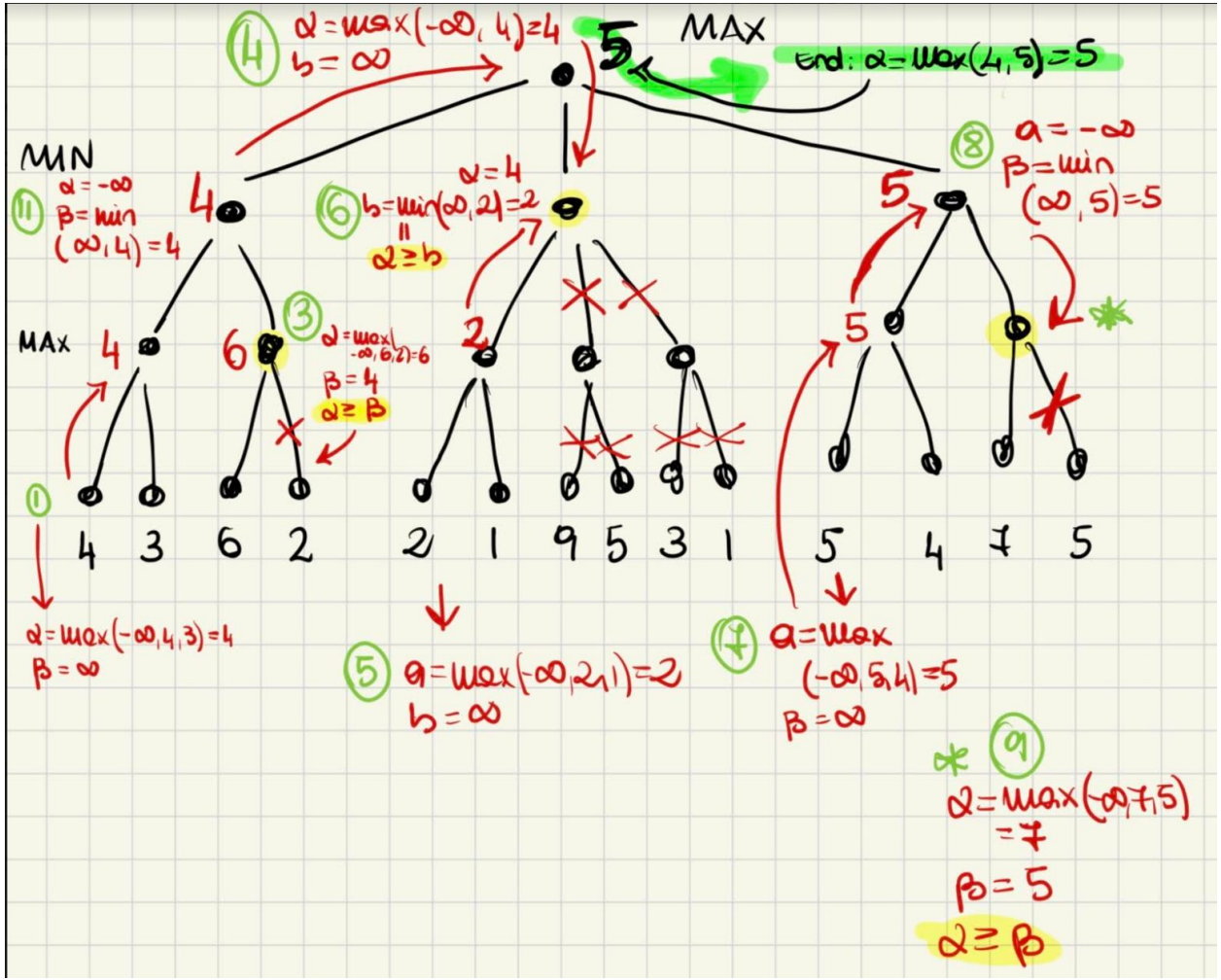
ALPHA = best value that the maximizer currently can guarantee at that level or above
Initialized with -infinity (the least value it can take, i.e. worst case scenario)

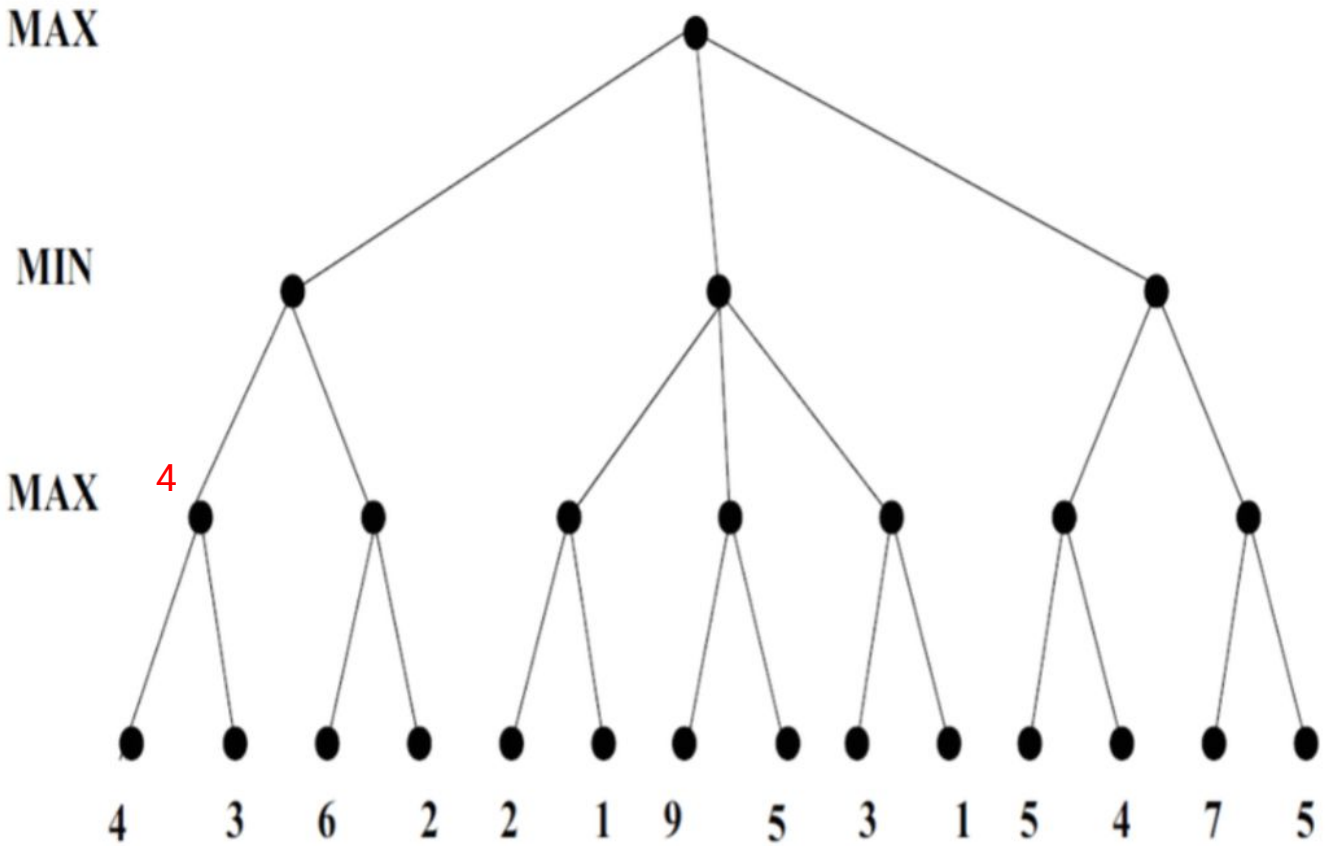
BETA = best value that the minimizer currently can guarantee at that level or above
Initialized with +infinity (the maximum value it can take, i.e. worst case scenario)

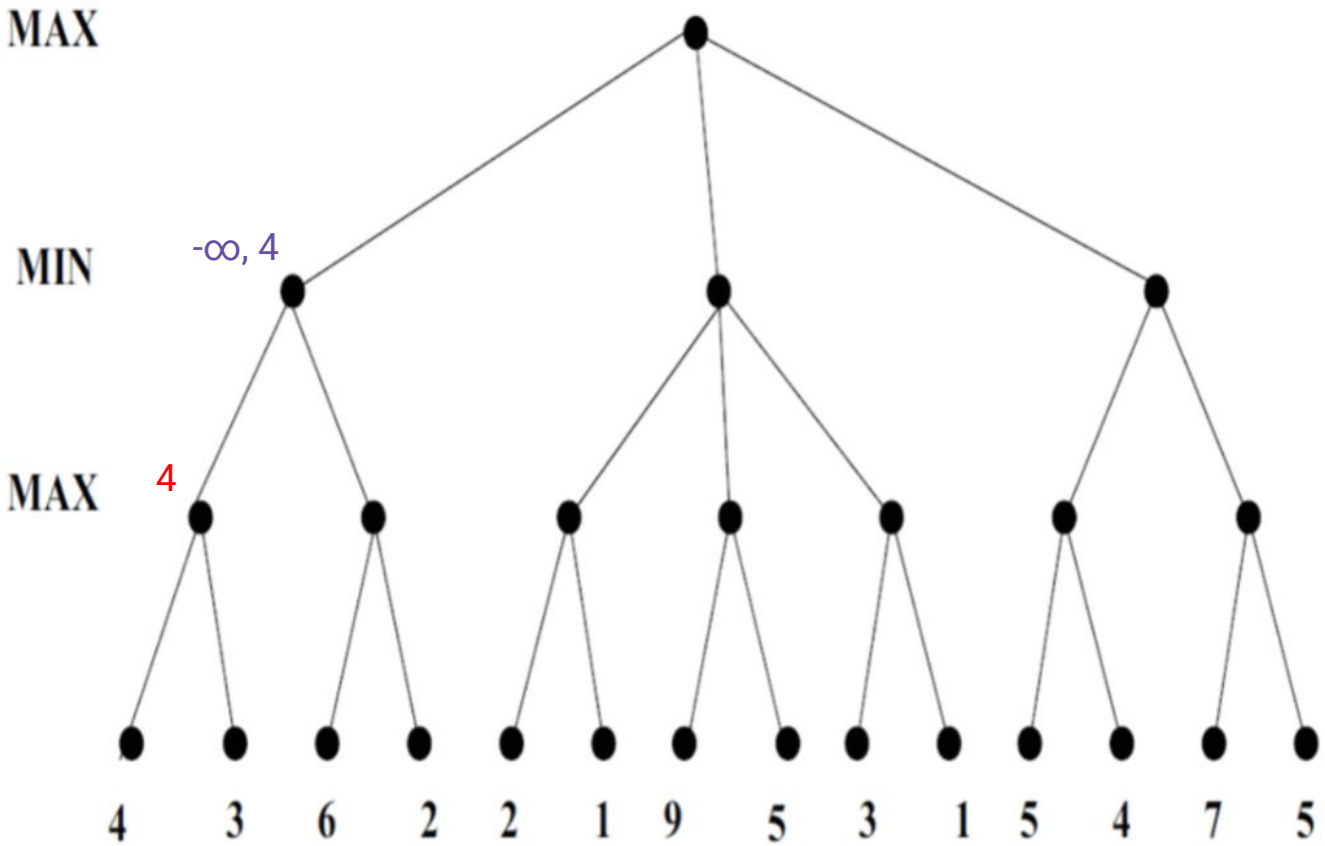
CONDITION FOR PRUNING:
Alpha >= Beta
When this condition is met at one of the levels, there is no need to investigate further nodes on that same level.

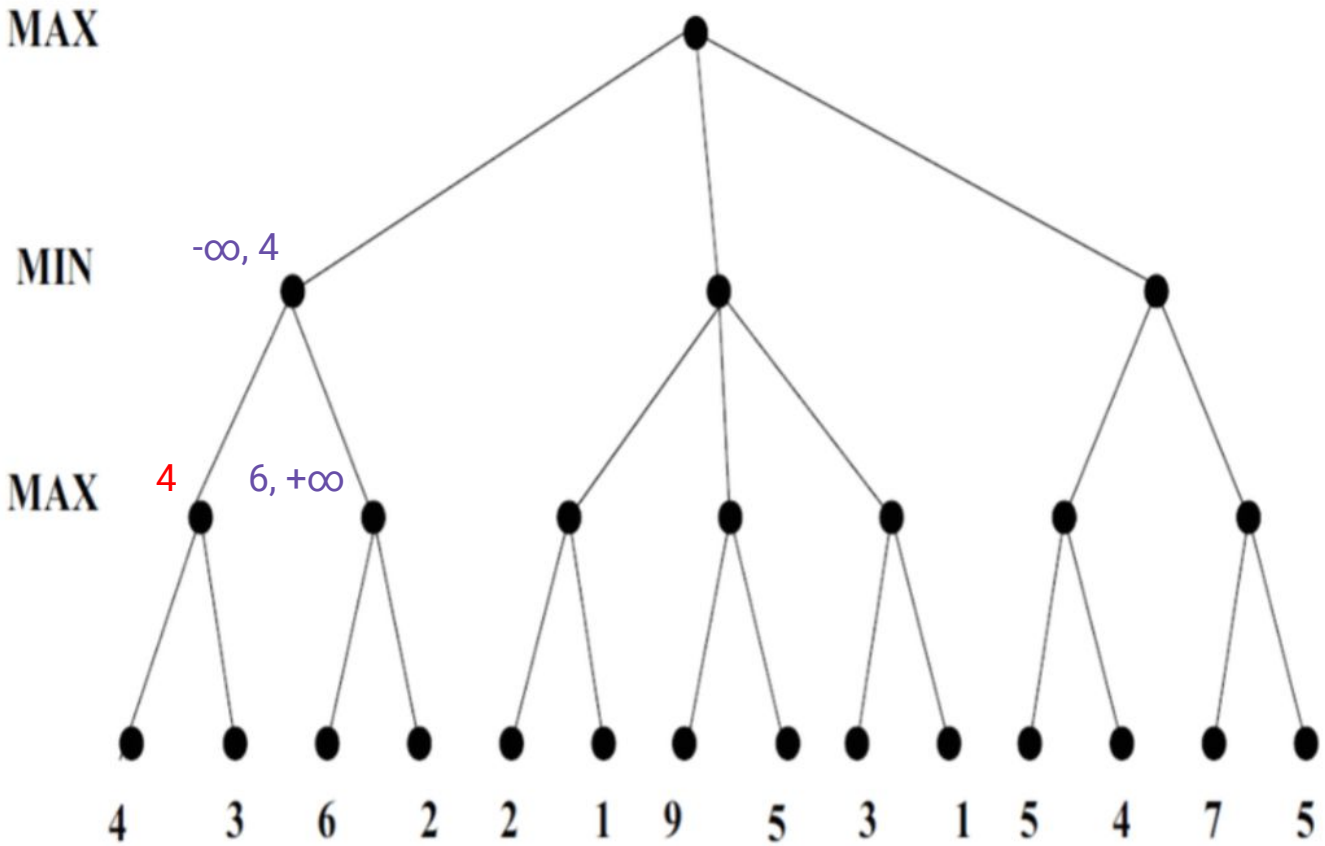
Exercise 8: Consider the following search tree that contains heuristic values at the leaves. The player on the move is depicted at each level. Which nodes can be ignored if we apply alpha-bèta pruning if the goal for Min is to have the smallest possible value, and for Max to have the largest possible value.

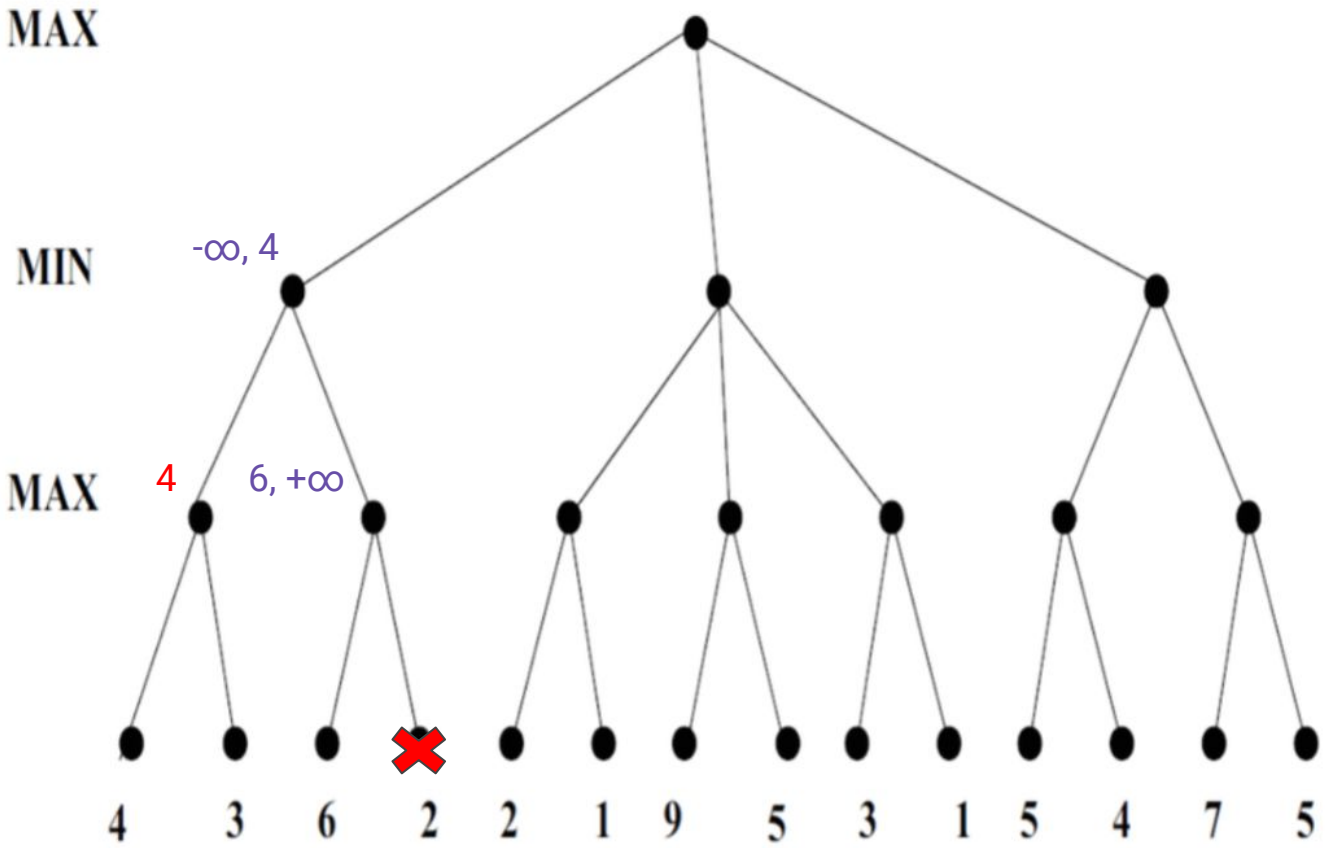


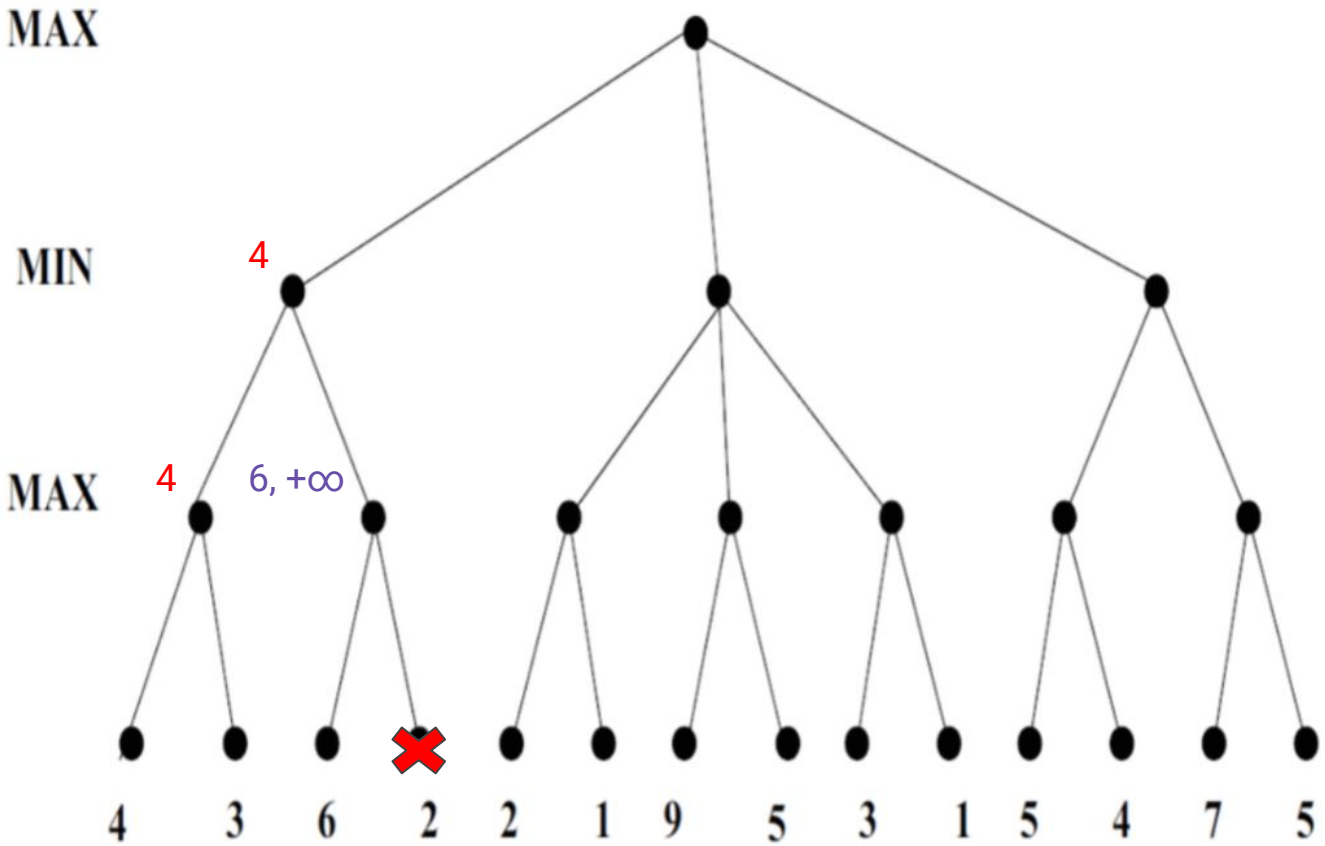

MAX
5
End: α = max(4, 5) = 5
4 α = max(-∞, 4) = 4
b = ∞
MIN
11 α = -∞
β = min
(∞, 4) = 4
4
6 b = min(∞, 2) = 2
α ≥ b
α = 4
2
8 α = -∞
β = min
(∞, 5) = 5
5
MAX
4
6
3 α = max(-∞, 6, 2) = 6
β = 4
α ≥ β
5
1
4 3 6 2 2 1 9 5 3 1 5 4 7 5
α = max(-∞, 4, 3) = 4
β = ∞
5 α = max
(-∞, 2, 1) = 2
b = ∞
7 α = max
(-∞, 5, 4) = 5
β = ∞
9
α = max(-∞, 7, 5)
= 7
β = 5
α ≥ β

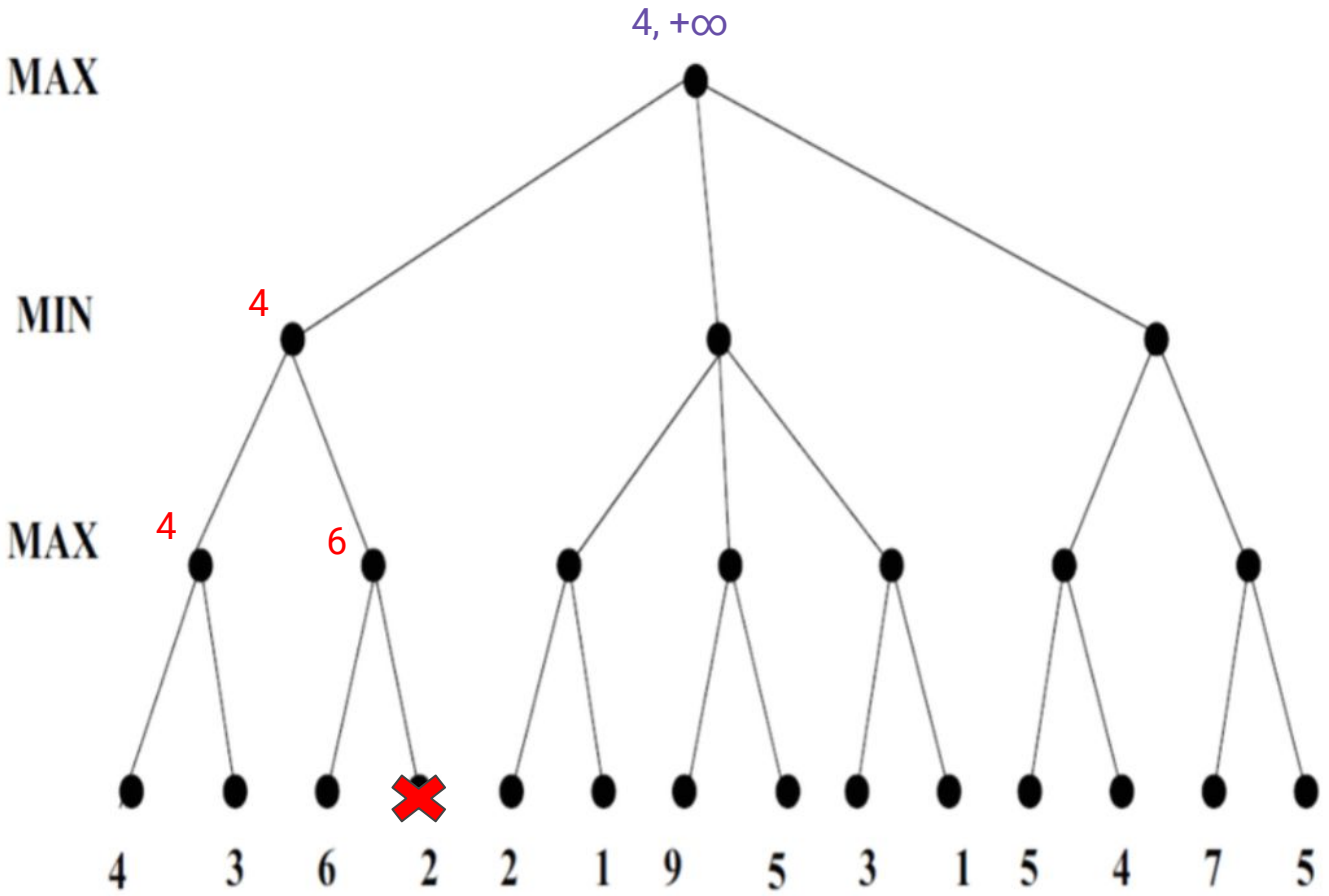

MAX
MIN
MAX
4
4 3 6 2 2 1 9 5 3 1 5 4 7 5

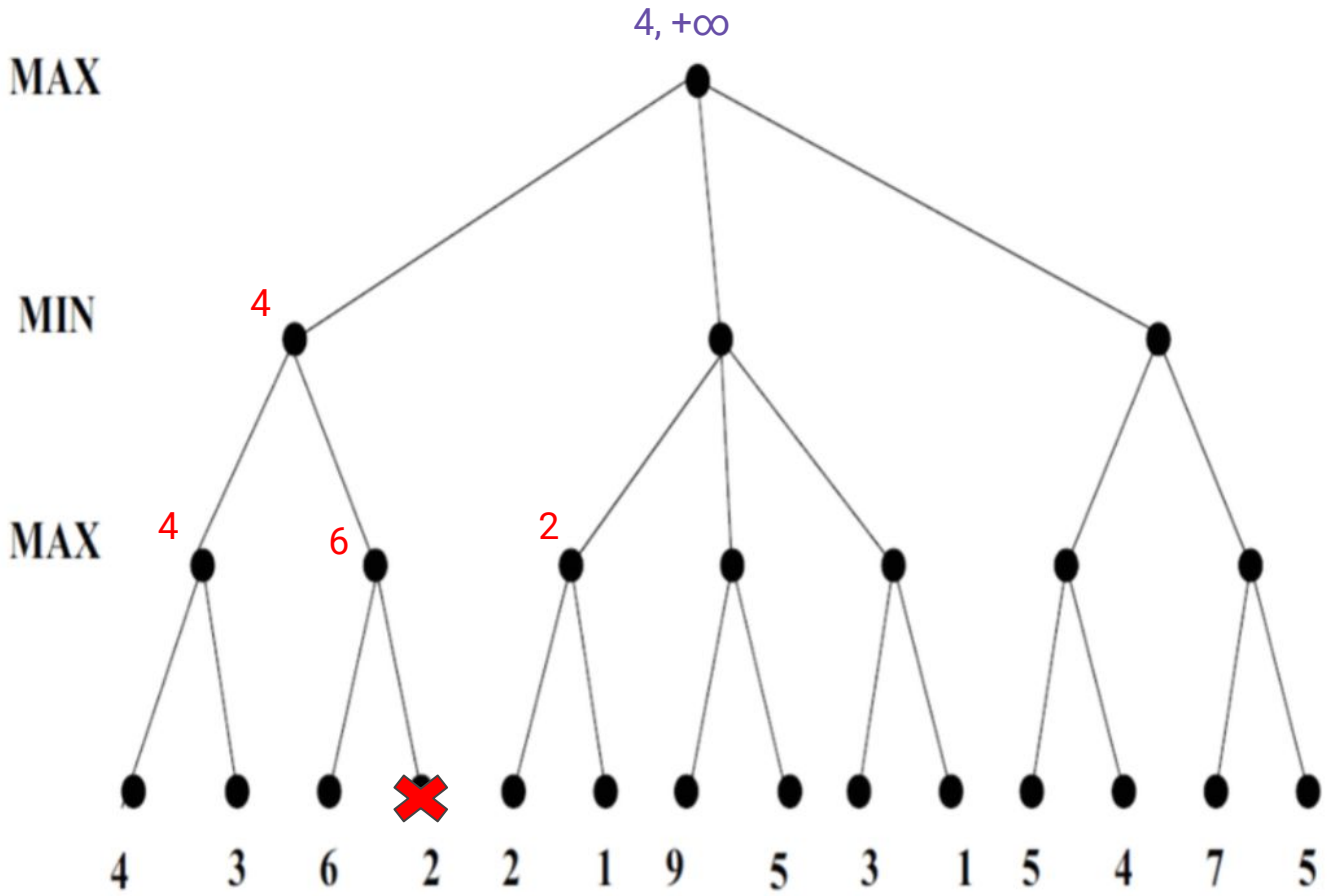

MAX
MIN
-∞, 4
MAX
4
4 3 6 2 2 1 9 5 3 1 5 4 7 5

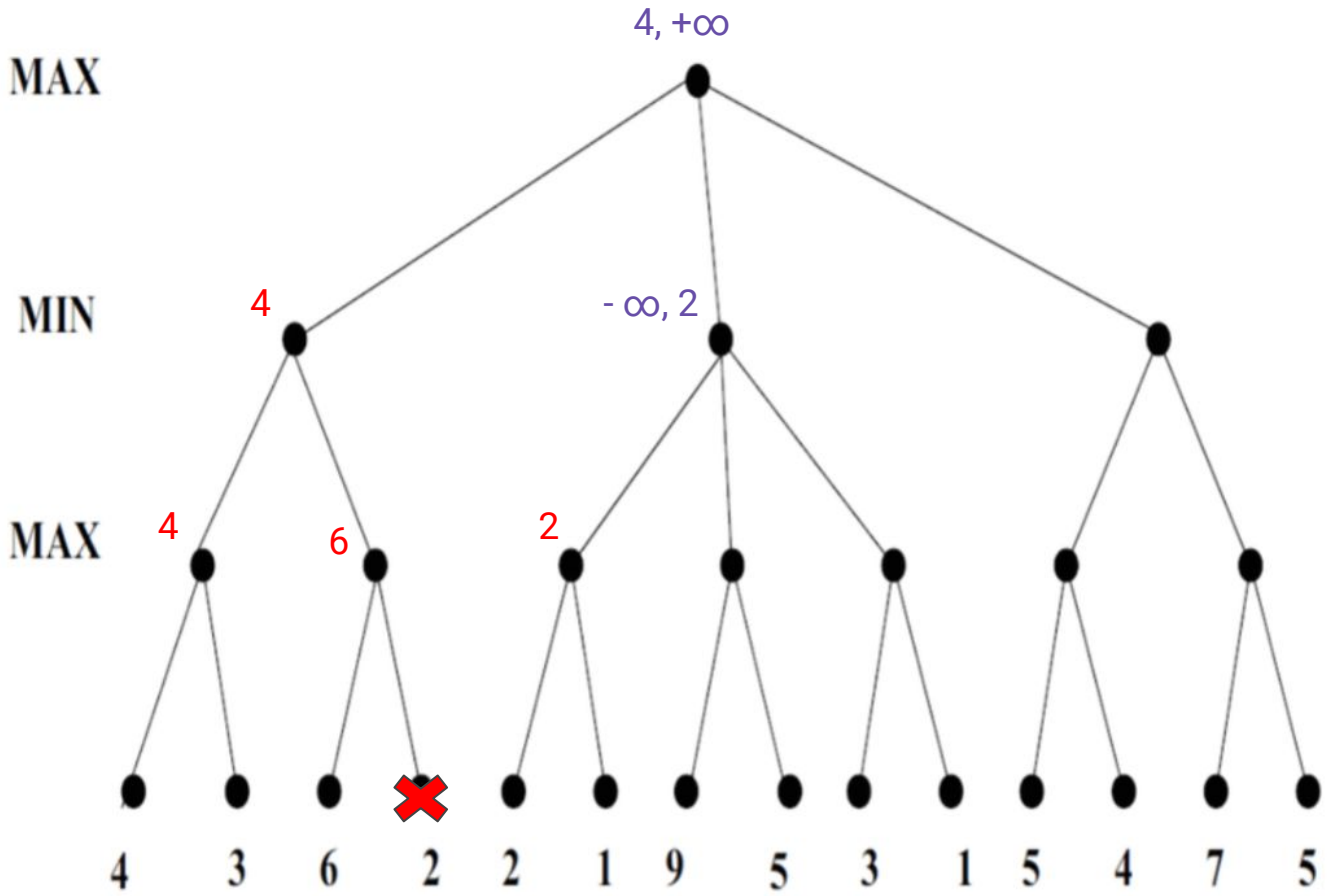

MAX
MIN
-∞, 4
MAX
4
6, +∞
4 3 6 2 2 1 9 5 3 1 5 4 7 5

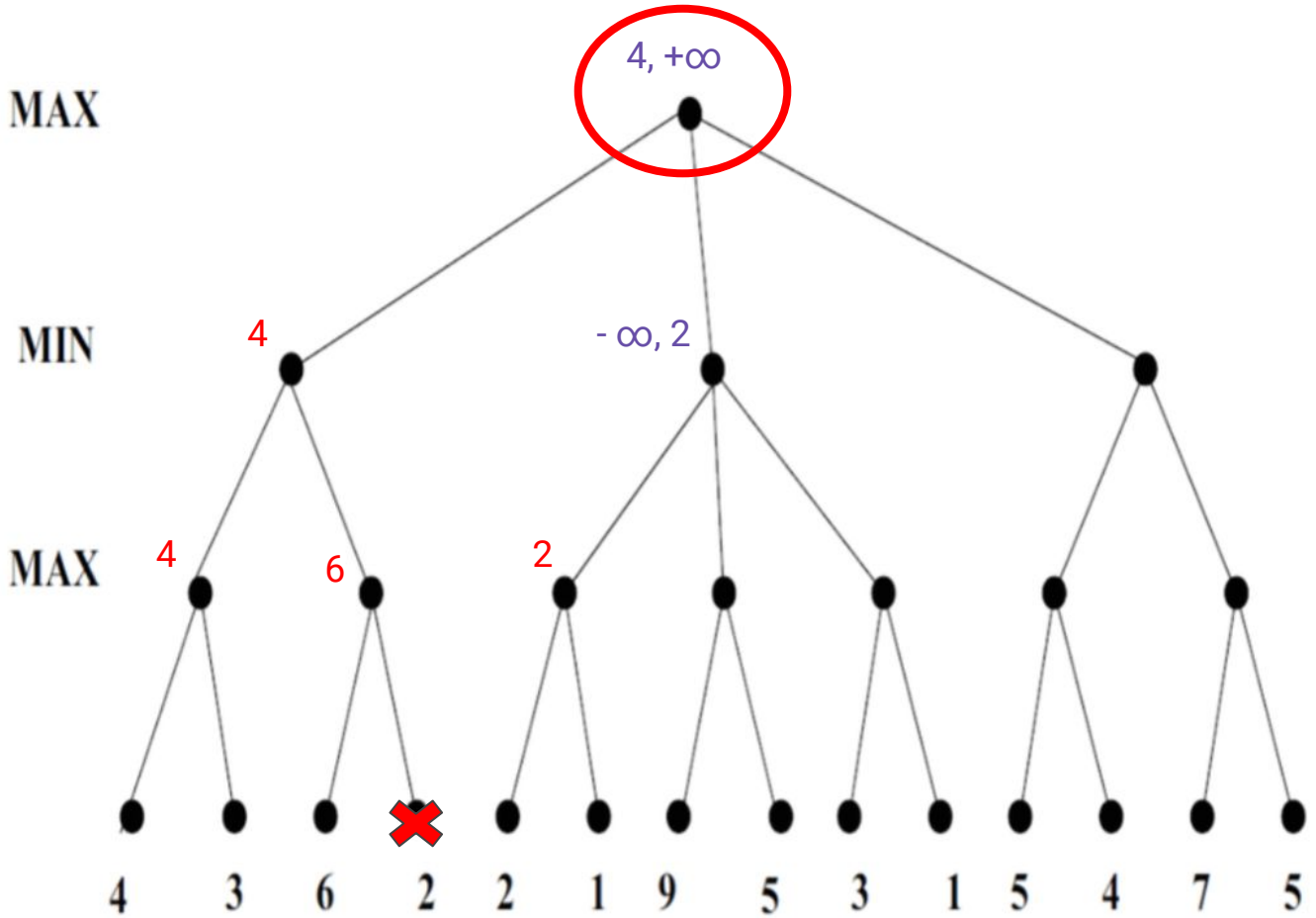

MAX
MIN
-∞, 4
MAX
4
6, +∞
4 3 6 2 2 1 9 5 3 1 5 4 7 5

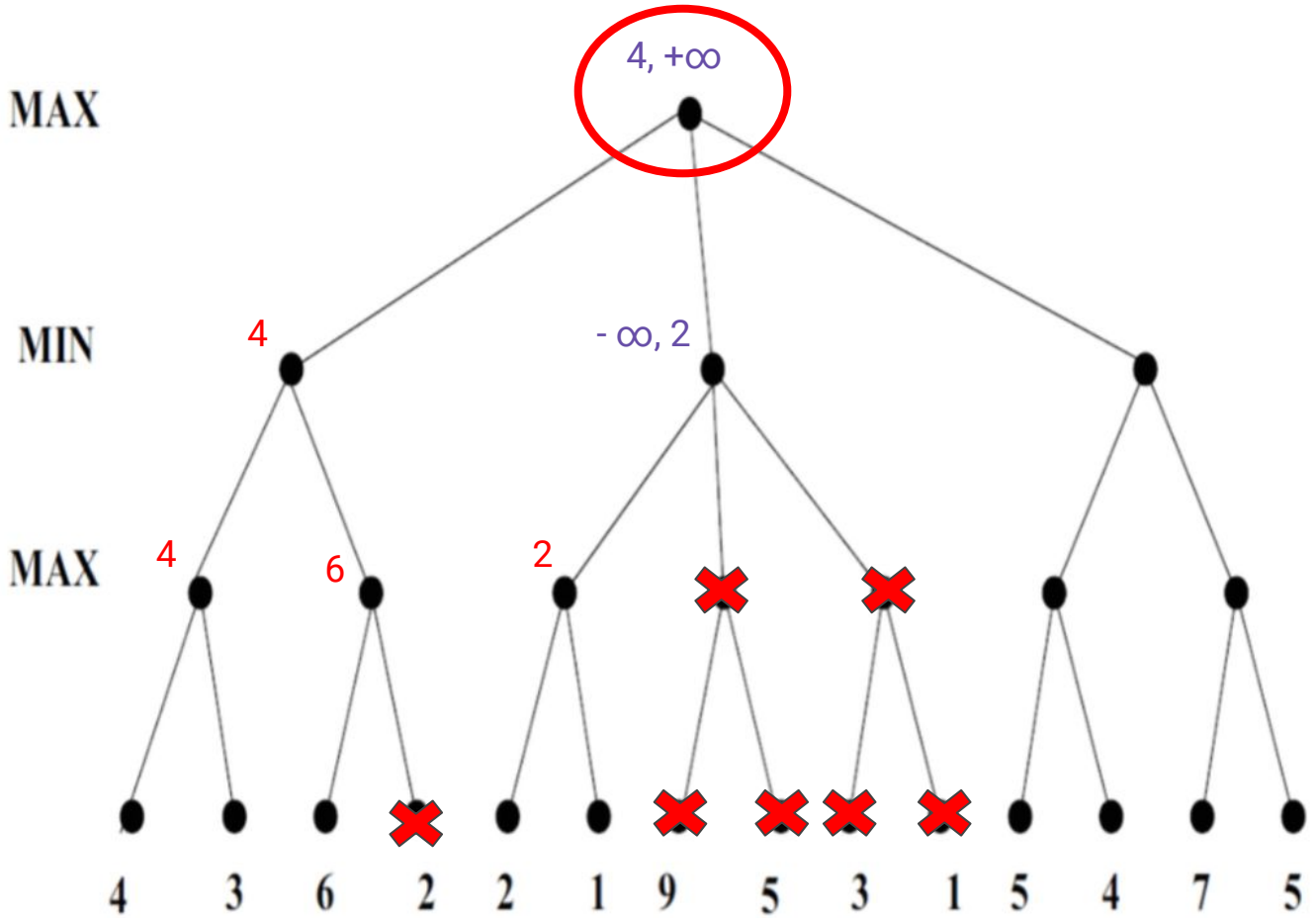

MAX
MIN
MAX
4
4
6, +∞
4 3 6 2 2 1 9 5 3 1 5 4 7 5

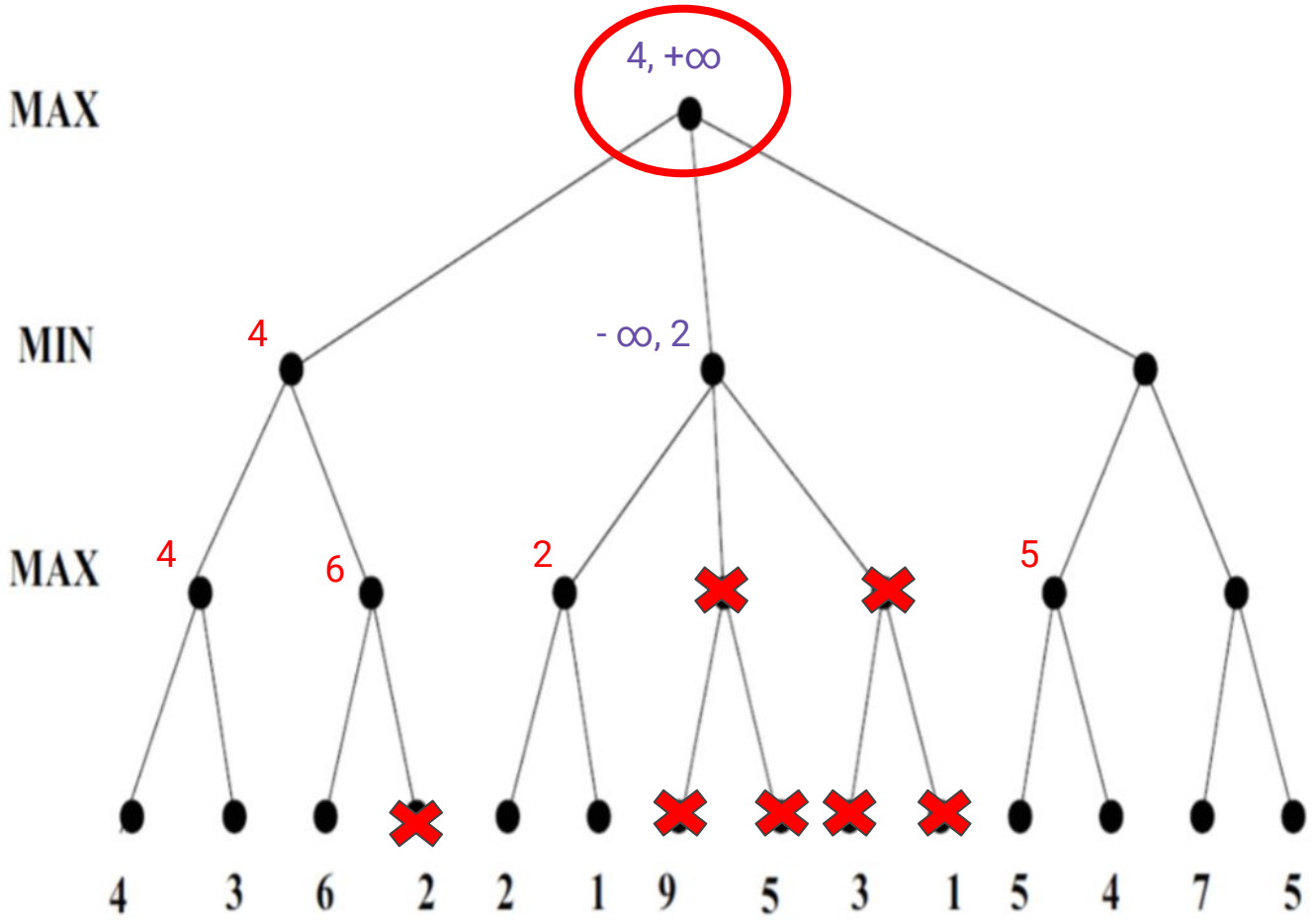

4, +∞
MAX
MIN
4
MAX
4
6
4 3 6 2 2 1 9 5 3 1 5 4 7 5

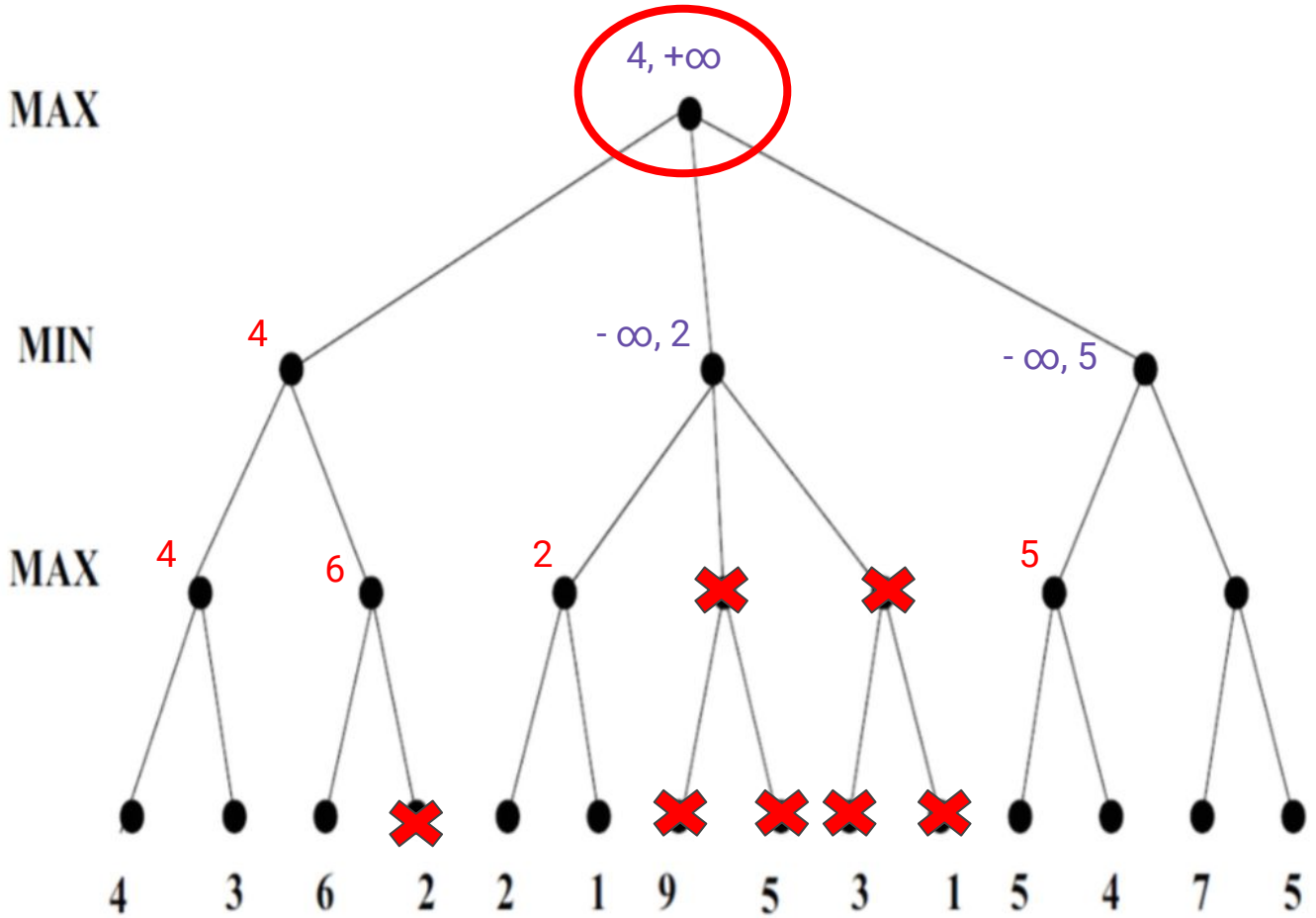

4, +∞
MAX
MIN
4
MAX
4
6
2
4 3 6 2 2 1 9 5 3 1 5 4 7 5

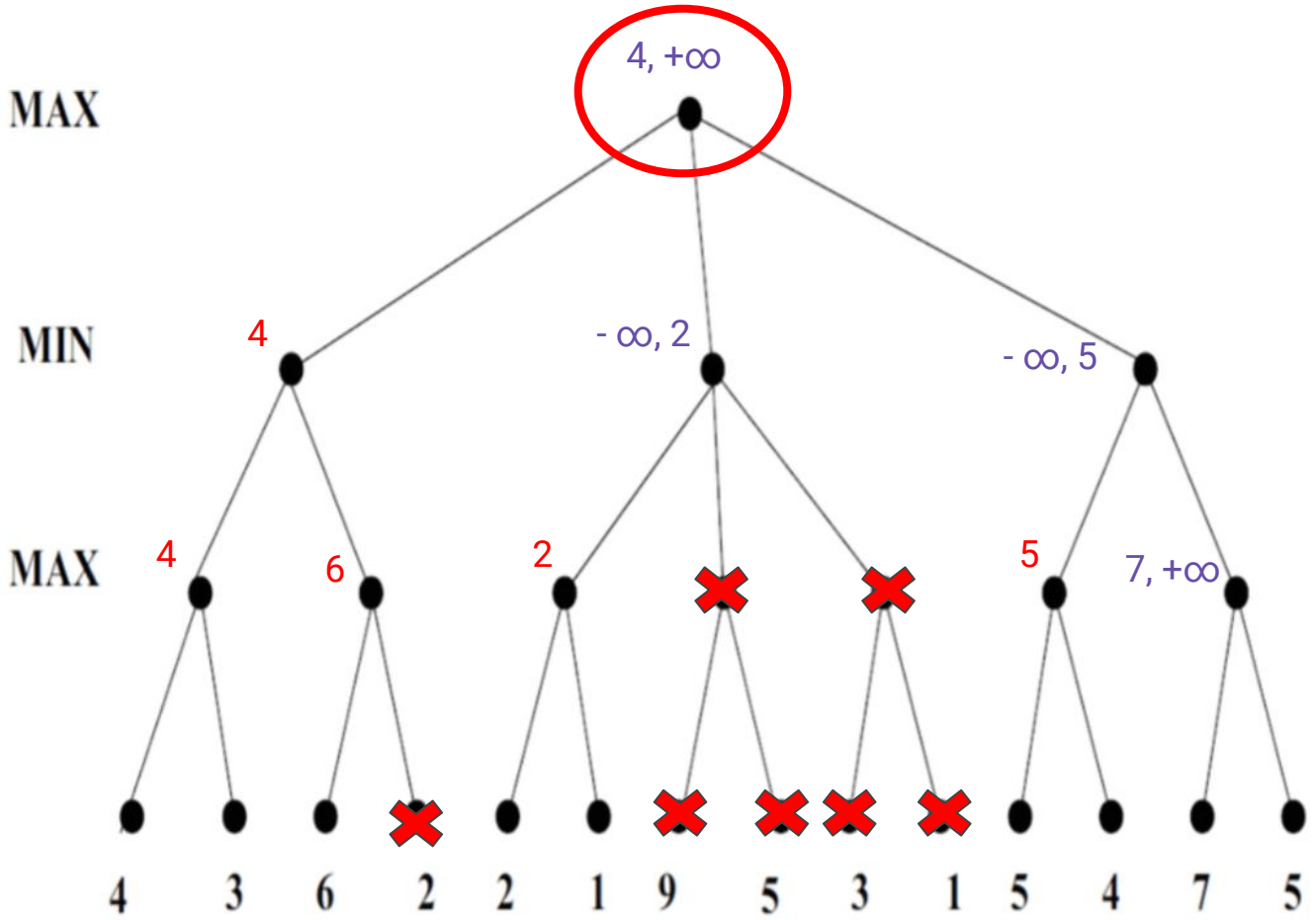

4, +∞
MAX
MIN
4
- ∞, 2
MAX
4
6
2
4 3 6 2 2 1 9 5 3 1 5 4 7 5

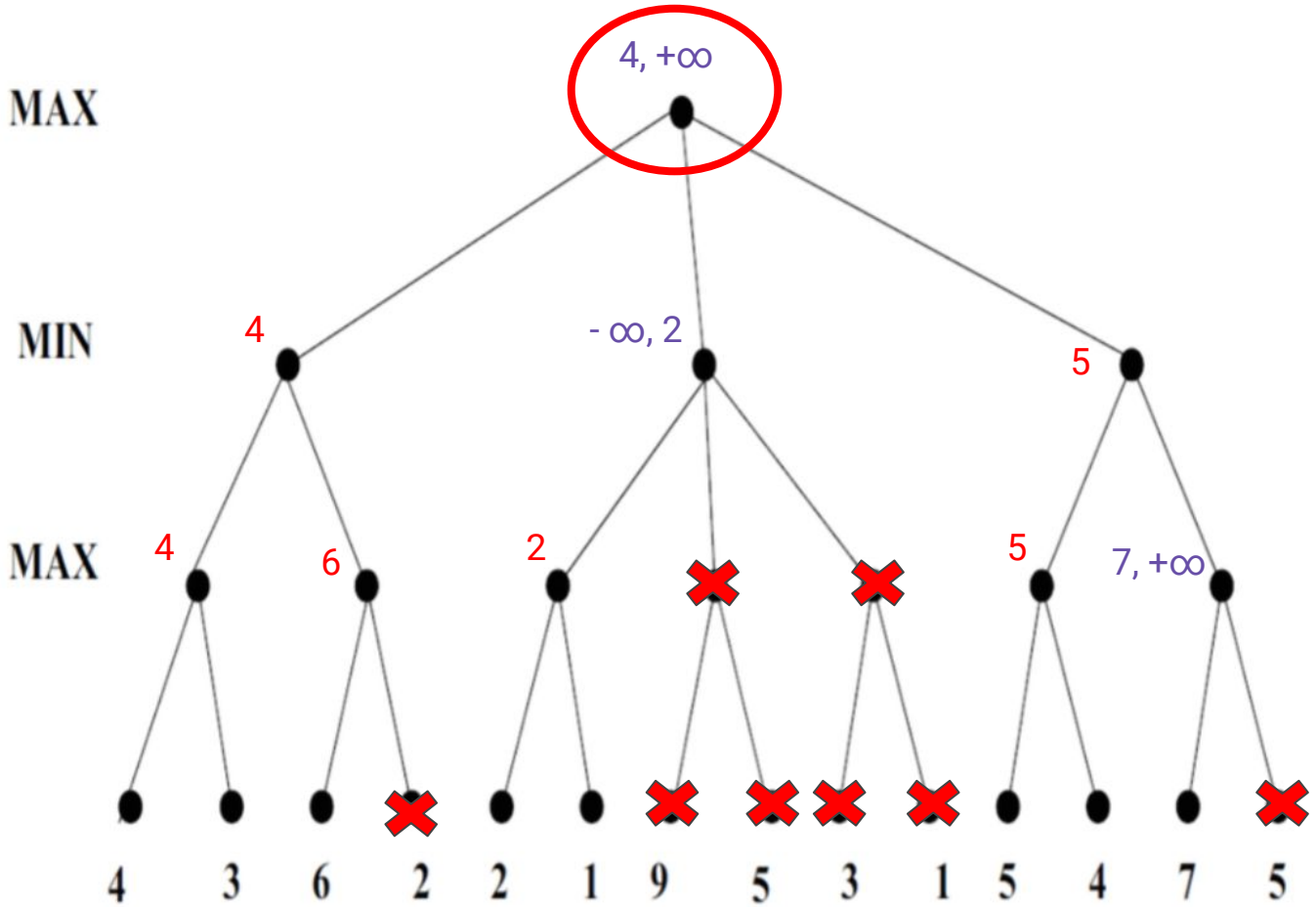

MAX
MIN
MAX
4, +∞
4
- ∞, 2
4
6
2
4 3 6 2 2 1 9 5 3 1 5 4 7 5

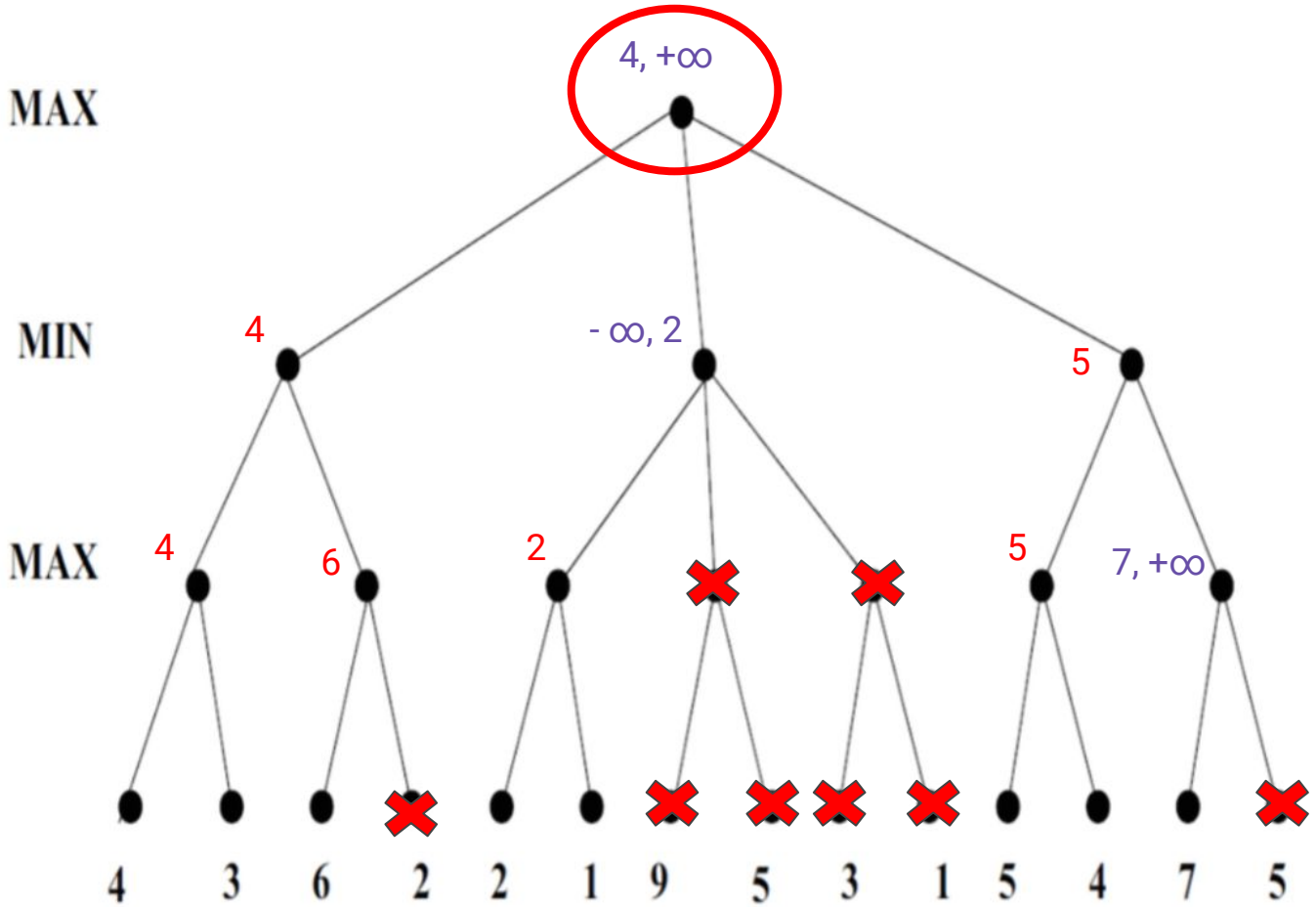

MAX
MIN
MAX
4, +∞
4
- ∞, 2
4
6
2
4 3 6 2 2 1 9 5 3 1 5 4 7 5

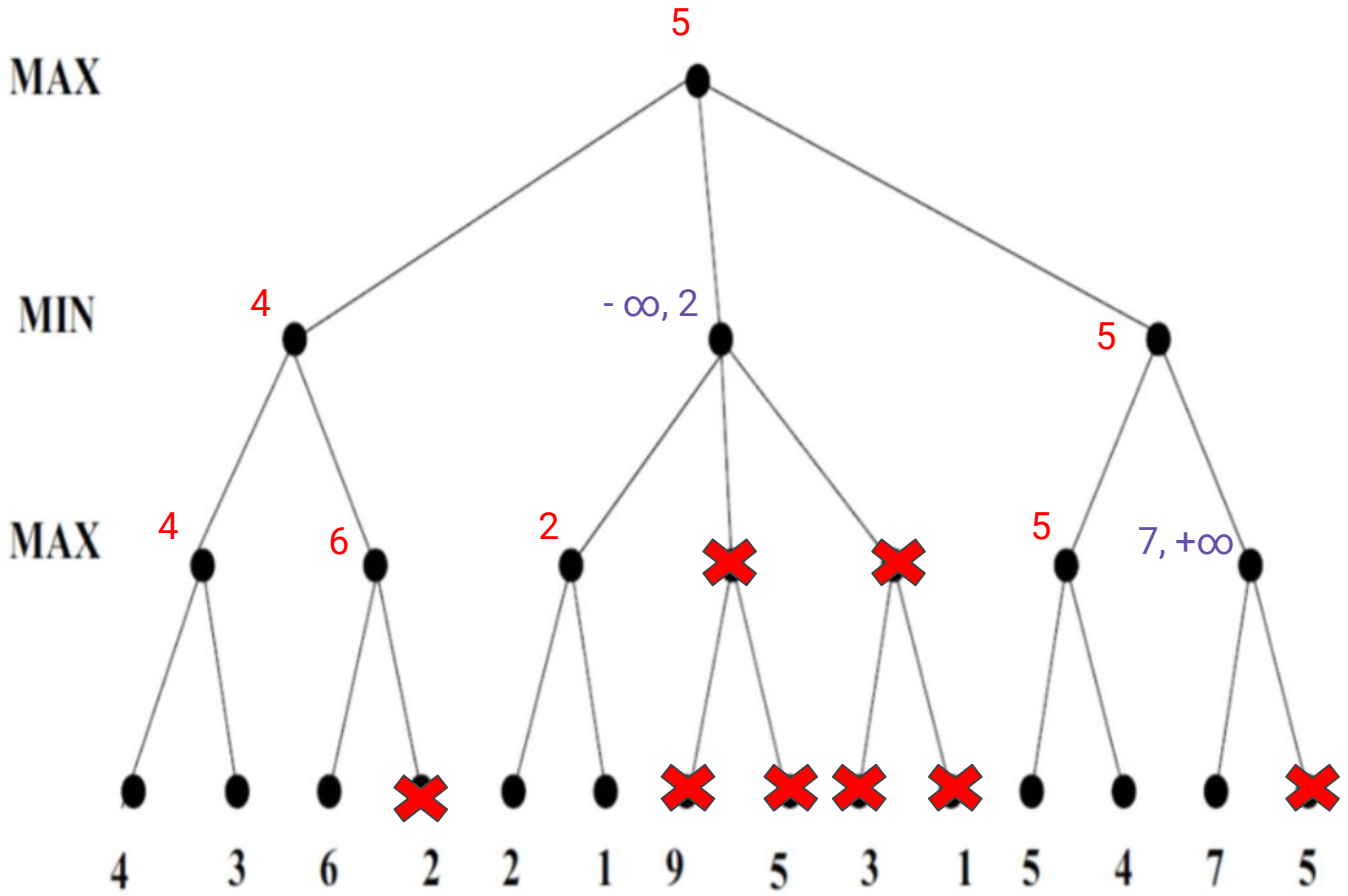

MAX
MIN
MAX
4, +∞
4
- ∞, 2
4
6
2
5
4 3 6 2 2 1 9 5 3 1 5 4 7 5


MAX
MIN
MAX
4, +∞
4
- ∞, 2
- ∞, 5
4
6
2
5
4 3 6 2 2 1 9 5 3 1 5 4 7 5


MAX
MIN
MAX
4, +∞
4
- ∞, 2
- ∞, 5
4
6
2
5
7, +∞
4 3 6 2 2 1 9 5 3 1 5 4 7 5


MAX
MIN
MAX
4, +∞
4
- ∞, 2
5
4
6
2
5
7, +∞
4 3 6 2 2 1 9 5 3 1 5 4 7 5


MAX
MIN
MAX
4, +∞
4
- ∞, 2
5
4
6
2
5
7, +∞
4 3 6 2 2 1 9 5 3 1 5 4 7 5


MAX
MIN
MAX
5
4
- ∞, 2
5
4
6
2
5
7, +∞
4 3 6 2 2 1 9 5 3 1 5 4 7 5

THE END